

Un perfil de integración para el intercambio gobernado de modelos de IA en espacios de datos: la ontología DAIMO

Journal Title
XX(X):1–24
©The Author(s) 2026
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Edmundo de Elvira Mori Orrillo¹ y Jiayun Liu¹

Abstract

El despliegue interorganizacional de modelos de inteligencia artificial exige un tratamiento semántico coherente de cinco tareas hoy fragmentadas entre vocabularios distintos: publicar el modelo como activo de catálogo, seleccionarlo bajo una política de uso, invocarlo mediante un servicio tipado, trazar cada ejecución y compararlo sobre condiciones de evaluación reproducibles. MLDCAT-AP describe el modelo, DCAT lo publica, ODRL expresa la política, PROV-O registra la procedencia y el *Dataspace Protocol* (DSP) negocia el intercambio contractual; ninguno articula las cinco tareas como cadena única.

Este artículo presenta DAIMO (*Data-space AI Model Ontology*), un perfil de integración OWL 2 DL que reutiliza los cinco vocabularios citados mediante alineamientos axiomáticos verificados. DAIMO introduce catorce clases nativas (nueve de primer nivel más cinco subclases de rol) que cubren los constructos ausentes en la unión de los vocabularios reutilizados, y se acompaña de una capa SHACL con restricciones estructurales, seis invariantes SHACL-SPARQL entre clases y un banco de pruebas negativas que confirma su carácter discriminante.

DAIMO se desarrolla siguiendo la metodología *Linked Open Terms* (LOT) y se ejercita sobre un caso de estudio multi-participante que instancia las veintitrés preguntas de competencia formuladas durante la especificación de requisitos. El paquete público incluye la ontología, la capa SHACL, el grafo de demostración, las consultas SPARQL y cinco scripts que reproducen los reportes de validación y el benchmark de escalabilidad acotada.

Keywords

gobernanza de modelos de IA; espacios de datos; alineamiento ontológico; validación SHACL; metodología LOT

Introducción

El Reglamento (UE) 2024/1689, conocido como *AI Act*, exige para los sistemas de inteligencia artificial de alto riesgo documentación técnica del modelo, registro de ejecuciones, trazabilidad del entrenamiento y evidencia de auditoría sostenida a lo largo del ciclo de vida⁴¹. Paralelamente, la iniciativa europea de espacios de datos, materializada en los conectores Eclipse Dataspace Components (EDC)³⁹ y en despliegues sectoriales como INESData o Gaia-X, consolida un modelo de intercambio contractual entre participantes autónomos sobre el *Dataspace Protocol* (DSP)³⁷. La conjunción de ambos marcos sitúa a los modelos de inteligencia artificial en un nuevo régimen: deben poder circular entre organizaciones como activos gobernados, con metadatos accionables que sostengan publicar, descubrir, invocar, trazar y comparar de forma coherente. Los vocabularios maduros de la Web Semántica cubren cada una de esas cinco tareas por separado, y este artículo argumenta que su integración en un único perfil es una contribución tanto necesaria como no trivial.

MLDCAT-AP 3.0.0, publicada por SEMIC en 2025³⁵, extiende DCAT para describir modelos de aprendizaje automático como activos de catálogo, con metadatos sobre tarea, algoritmo, ejecuciones, evaluaciones, riesgos y recursos computacionales. DCAT² aporta las primitivas de catálogo (*Catalog*, *Dataset*, *Distribution*,

DataService, *CatalogRecord*) sobre las que MLDCAT-AP se apoya. ODRL⁴ proporciona un modelo consolidado de ofertas, permisos, prohibiciones y acuerdos, y es la base natural para expresar las políticas de uso que gobiernan la invocación del modelo. PROV-O³ es el estándar para procedencia y atribución de entidades, actividades y agentes. DSP³⁷ especifica los mensajes de negociación de contratos y transferencia que EDC implementa. Las *model cards*⁶, *factsheets*⁷ y *datasheets*¹⁹ complementan estos vocabularios con descripciones narrativas del uso previsto, las limitaciones y las condiciones de entrenamiento.

Cada uno de estos vocabularios resuelve bien su parte, pero los huecos entre ellos son precisamente los que un perfil de integración debe cerrar. Un modelo publicado como `it6:MachineLearningModel` de MLDCAT-AP no lleva adherido el acto de ser ofrecido a través de una frontera organizativa. Una `odrl:Offer` no está anclada a un modelo concreto dentro de un catálogo federado. Una negociación DSP es agnóstica respecto al tipo de activo

¹Departamento de Inteligencia Artificial, Escuela Técnica Superior de Ingenieros Informáticos, Universidad Politécnica de Madrid, España

Corresponding author:

Edmundo de Elvira Mori Orrillo, Universidad Politécnica de Madrid, Campus de Montegancedo, 28660 Boadilla del Monte, Madrid, España.
Email: edmundo.mori.orrillo@upm.es

negociado y no conoce que el intercambio versa sobre un modelo de IA con métricas y contextos de evaluación que condicionan su uso. Un `prov:Bundle` de PROV-O no agrupa por sí solo las actividades ocurridas bajo contextos administrativos distintos en torno a un mismo artefacto derivado. Las cinco tareas operativas del intercambio exigen articular estos huecos de forma coherente.

No existe todavía una ontología reutilizable que vincule metadatos de publicación en catálogo, políticas de acceso, contratos de invocación, trazas de ejecución, evidencia de auditoría y evaluación comparativa para modelos de IA que circulan entre participantes de un espacio de datos, con alineamientos axiomáticos verificados contra los vocabularios existentes y con evidencia ejecutable y reproducible de su corrección. Cerrar esa distancia exige dos principios simultáneos: reutilizar lo que los vocabularios maduros ya resuelven y añadir sólo los constructos que la articulación entre ellos requiere.

Este artículo presenta DAIMO (*Data-space AI Model Ontology*) y aporta cuatro contribuciones.

La **primera** es la propia ontología de integración: un perfil OWL 2 DL que reutiliza DCAT-AP, MLDCAT-AP 3.0.0, ODRL 2.2, PROV-O y DSP mediante alineamientos axiomáticos `rdfs:subClassOf` y `rdfs:subPropertyOf`, no simples listas de prefijos. Sobre esa capa de reutilización, DAIMO introduce catorce clases nativas que cubren los constructos ausentes en la unión de los vocabularios reutilizados. Cada clase nativa se justifica por un hueco concreto y por al menos una pregunta de competencia que no puede responderse sin ella.

La **segunda** es un mecanismo de *verificación de implicación* sobre el cierre OWL-RL que complementa al razonamiento estándar de consistencia. Para cada clase nativa, la comprobación enumera las superclases inferidas y las contrasta con una lista de superclases prohibidas semánticamente. Este mecanismo detecta alineamientos que un razonador DL acepta sin contradicción pero que introducen tipados no intencionados bajo razonamiento.

La **tercera** es una capa SHACL que integra seis invariantes SPARQL entre clases con un banco de pruebas negativas. Cada invariante dispone de un caso diseñado para violarla. La validación conforma sobre el grafo positivo y dispara las seis violaciones sobre el grafo negativo, lo que demuestra que las restricciones son discriminantes y no meramente satisficibles.

La **cuarta** es un caso de estudio multi-participante que instancia la ontología sobre un escenario de predicción de riesgo de inundación entre una universidad, un ayuntamiento, un evaluador científico y una plataforma federada de espacios de datos. El caso actúa como demostrador: ejerce cada clase nativa al menos una vez y hace que las veintitrés consultas de competencia devuelvan las filas esperadas.

Trabajo relacionado

DAIMO se sitúa en la intersección de cuatro familias de trabajo que la literatura existente trata habitualmente por separado: documentación narrativa de modelos, ontologías del ciclo de vida de aprendizaje automático, vocabularios de gobernanza y estándares de espacios de datos. Revisamos

cada familia por orden de proximidad al problema, identificando qué aporta al puente catálogo-dataspace, dónde lo deja abierto y cómo DAIMO se posiciona respecto a ella. Una quinta subsección revisa las ontologías SWJ recientes cuya metodología de validación hemos adoptado. La Tabla 2, inspirada en la matriz de reuso de RePlanIT⁹, compara todas las familias contra los cinco atributos operacionales que un perfil para modelos de IA gobernados debe cubrir.

Descripción de modelos de IA

Documentación narrativa. Las *model cards* de Mitchell et al.⁶, las *factsheets* de Arnold et al.⁷ y las *datasheets for datasets* de Gebru et al.¹⁹ elevaron los estándares de transparencia sobre artefactos de IA. Describen en lenguaje natural el uso previsto del modelo, sus limitaciones, el rendimiento esperado sobre benchmarks conocidos y las condiciones bajo las que ha sido entrenado y evaluado. Son valiosas para la comunicación con lectores humanos y para la rendición de cuentas cualitativa, pero insuficientes como sustrato para descubrimiento automatizado, filtrado por política o vinculación de evidencia en flujos máquina a máquina: un espacio de datos requiere primitivas que los agentes de software puedan consumir, no documentos que otros agentes deban interpretar.

Ontologías del ciclo de vida ML. Allí donde la documentación narrativa falla en el consumo máquina, las ontologías estructuradas del ciclo de vida ML ocupan su lugar. EXPO²² ofrece una ontología general de experimentos científicos que ha servido de base a trabajos posteriores. ML-Schema²³ formaliza la semántica de experimentos de aprendizaje automático mediante un esquema reutilizable y ligero. MEX²⁴ provee un vocabulario de intercambio para resultados de experimentos, y OntoDM²⁵ junto con DMOP²⁶ articulan el dominio de la minería de datos y el *semantic meta-mining*. Souza et al.³⁰ y Schmidt et al.³¹ extienden la perspectiva hacia la procedencia y la FAIRness del ciclo de vida ML. Estas ontologías proporcionan un fundamento riguroso pero permanecen centradas en el experimento científico y en el proceso de entrenamiento; ninguna aborda la dimensión contractual del intercambio entre participantes autónomos.

Un grupo reciente de trabajos se desplaza desde el experimento hacia la taxonomía y gestión de activos de IA. La *Artificial Intelligence Ontology* (AIO) sistematiza conceptos técnicos y éticos de IA en ramas como sesgo, capas, tareas de aprendizaje automático, funciones matemáticas, modelos, redes, preprocesamiento y estrategias de entrenamiento²⁷. La *Intelligence Task Ontology* (ITO) organiza tareas, benchmarks y métricas de rendimiento para analizar el progreso de capacidades de IA a gran escala²⁸. En la misma línea, Novacek et al. proponen una gestión ontológica de modelos y datasets orientada a relacionar modelos con los datos usados para su entrenamiento y validación²⁹. Estos enfoques refuerzan la necesidad de describir la estructura técnica del modelo, sus datos y sus métricas; DAIMO aborda una pregunta distinta y complementaria: cómo convertir esas descripciones en un activo ofertable, autorizable, invocable, auditable y comparable dentro de un espacio de datos federado.

El trabajo más próximo al dominio de DAIMO, y el más reciente, es MLDCAT-AP 3.0.0³⁵, publicado por SEMIC en 2025. MLDCAT-AP extiende DCAT para describir modelos de IA como subclases de `dcat:Dataset`, con metadatos ricos sobre política, benchmark, evaluación, ejecución, riesgo e infraestructura computacional. DAIMO lo reutiliza directamente: la clase `it6:MachineLearningModel` es la clase del modelo en DAIMO, y las clases `it6:Task`, `it6:Run`, `it6:Flow`, `it6:Evaluation` e `it6:ComputerInfrastructure` capturan respectivamente la tarea objetivo, la ejecución concreta, el *pipeline* asociado, la medición resultante y el entorno de cómputo. MLDCAT-AP es fuerte en el lado del modelo pero deliberadamente agnóstico respecto al espacio de datos en el que el modelo circula: no modela el acto de ofrecer un modelo a través de fronteras organizativas, ni las nociones de despliegue gobernado, autorización de ejecución anclada a un acuerdo o procedencia agregada entre contextos administrativos distintos. MLDCAT-AP cubre el modelo; DAIMO cubre el puente entre el modelo y el dataspace.

La Tabla 1 explicita la relación clase a clase entre DAIMO y MLDCAT-AP. El patrón es intencional: DAIMO adopta las clases de MLDCAT-AP para el plano del activo de IA, añade restricciones de perfil cuando el intercambio gobernado exige completitud operativa, e introduce extensiones sólo para las relaciones puente que MLDCAT-AP no cubre. La versión del paquete validado se mantiene sobre MLDCAT-AP 3.0.0 porque los alineamientos, ejemplos y pruebas SHACL fueron contruidos contra esa especificación. MLDCAT-AP 3.1.0, publicada como *Candidate Recommendation* en 2026³⁶, debe revisarse antes del envío final para confirmar que la tabla no cambia o, en caso contrario, actualizar el módulo de alineamientos y reejecutar la suite de validación.

Gobernanza de recursos y espacios de datos

Vocabularios W3C de gobernanza. Las ontologías del ciclo de vida describen qué es un modelo y cómo se produjo, pero no gobiernan cómo se publica, se licencia o se atribuye. Tres vocabularios W3C proporcionan ese sustrato de gobernanza. DCAT 2² proporciona las primitivas de catálogo sobre las que se apoyan tanto MLDCAT-AP como DAIMO. ODRL 2.2⁴ ofrece un modelo consolidado para expresar permisos, prohibiciones, obligaciones, ofertas (`odrl:Offer`) y acuerdos (`odrl:Agreement`) vinculantes; DAIMO lo usa intensivamente tanto para la política adherida a las ofertas como para los acuerdos que autorizan ejecuciones concretas. PROV-O³ aporta la ontología de procedencia con actividades, entidades, agentes, bundles y roles, y es la capa natural sobre la que codificar la trazabilidad de ejecuciones individuales y su agregación entre participantes. Pandit y Esteves¹⁵ muestran cómo ODRL se combina con DPV para escenarios de salud, un patrón que DAIMO transpone al dominio IA. Estos vocabularios son maduros y ampliamente adoptados, pero por construcción son agnósticos al dominio: ninguno es específico de inteligencia artificial ni de espacios de datos.

Estándares de espacios de datos. Mientras que los vocabularios anteriores describen artefactos, los estándares de espacios de datos describen el intercambio en sí. La

perspectiva desplaza el foco desde los artefactos aislados hacia los intercambios gobernados entre actores autónomos. Franklin et al.¹ formularon las *dataspace*s como respuesta a la heterogeneidad persistente de fuentes. Otto et al.¹⁷ y Cappelletto et al.¹⁸ convierten la soberanía del dato, los contratos y la interoperabilidad controlada en supuestos explícitos del diseño. El *Dataspace Protocol*³⁷ es la especificación W3C-CG que define los mensajes de catálogo, la negociación de contratos y los procesos de transferencia; Eclipse Dataspace Components³⁹ es su implementación de referencia.

Conviene distinguir entre *framework* y *vocabulario*: EDC es un framework de ejecución construido en Java, no una ontología. La semántica que EDC implementa proviene de DSP (protocolo), ODRL (políticas) y DCAT (catálogo); sólo un puñado de conceptos son verdaderas extensiones específicas de EDC, y de ellos DAIMO referencia explícitamente `edc:ParticipantContext` por su utilidad para anclar la procedencia agregada a un contexto administrativo. Esta aclaración es importante porque en la literatura de espacios de datos las dos capas se confunden con frecuencia. La versión validada del artefacto DAIMO conserva los términos DSP utilizados por el paquete actual; antes de la sumisión final debe contrastarse con la release estable DSP 2025-1³⁸ y, si cambia la semántica de negociación o transferencia relevante, actualizar las notas de alineamiento informativo. Trabajos sobre ecosistemas federados como ConSolid⁸ muestran además que la Web Semántica evoluciona hacia escenarios multi-actor donde gobernanza e interoperabilidad deben diseñarse conjuntamente.

Ingeniería ontológica ejecutable en SWJ

Más allá del sustrato de dominio que las cuatro familias anteriores proporcionan, un quinto hilo de trabajo reciente informa cómo debe evaluarse DAIMO. El *Semantic Web Journal* ha publicado recientemente un conjunto de ontologías que comparten un compromiso metodológico explícito con la validación ejecutable y la reproducibilidad. Kurteva et al. presentan RePlanIT⁹, una ontología para pasaportes digitales de producto de equipos ICT, con SHACL y evaluación de usabilidad con expertos de industria. Palma et al. operacionalizan información de suelos a escala global en GloSIS¹⁰ mediante un perfil modular alineado con SOSA/SSN e ISO 28258. Woods et al. conectan ingeniería ontológica con procesamiento del lenguaje natural para actividades de mantenimiento industrial¹¹. Bareedu et al. derivan reglas SHACL desde estándares industriales OPC UA¹³. Ferranti et al. formalizan restricciones de propiedad de Wikidata mediante SHACL y SPARQL¹⁴. Robaldo y Batsakis¹⁶ analizan la interacción entre validación SHACL e inferencia sobre la Ontología del Tiempo. Penteado et al.¹² estudian metodologías de publicación de linked open government data. Estos trabajos establecen colectivamente la expectativa —que DAIMO adopta como vinculante— de que una ontología con pretensiones operacionales presente evidencia reproducible de su corrección, no solamente una descripción axiomática.

La lectura horizontal de la Tabla 2 indica que ningún artefacto individual cubre simultáneamente las cinco dimensiones; la lectura vertical confirma que cada columna

Tabla 1. Relación entre DAIMO y MLDCAT-AP 3.0.0, clase por clase. *Adoptada* indica que el constructo permanece en la capa MLDCAT-AP; *perfilada*, que DAIMO añade expectativas de conformidad; *extensión puente*, que DAIMO introduce un constructo para la relación catálogo-dataspace ausente en MLDCAT-AP.

Clase MLDCAT-AP	Tratamiento DAIMO	Notas
it6:MachineLearningModel	Adoptada + perfilada	Descripción del activo de IA; DAIMO exige política, título e identificador para intercambio gobernado.
it6:Task	Adoptada	Tarea objetivo del modelo.
it6:Run	Adoptada + perfilada	Registro de ejecución; DAIMO exige <i>flow</i> , algoritmo, agente, marca temporal y enlace de autorización.
it6:Flow	Adoptada	Pipeline asociado a una ejecución.
it6:Evaluation	Adoptada	Resultado de medición; puede formar parte de la procedencia cruzada.
it6:EvaluationMeasure	Adoptada	Tipo y valor de métrica.
it6:Benchmark	Adoptada	Referencia de benchmark para condiciones de comparabilidad.
it6:ComputerInfrastructure	Adoptada	Entorno de cómputo donde se ejecuta un despliegue.
it6:Hardware, it6:Library	Adoptadas	Componentes del entorno de cómputo.
it6:HarmRisk	Adoptada	Anotación de riesgo sobre el modelo.
it6:Modality	Adoptada	Modalidad de entrada.
it6:trainedOn, it6:testedOn	Adoptadas	Enlaces a datasets de entrenamiento y prueba.
Constructos puente ausentes en MLDCAT-AP y añadidos por DAIMO		
(sin análogo MLDCAT-AP)	Extensión puente	Oferta de catálogo gobernada con política ODRL asociada.
(sin análogo MLDCAT-AP)	Extensión puente	Autorización de ejecución basada en un acuerdo.
(sin análogo MLDCAT-AP)	Extensión puente	Relación de despliegue entre modelo, infraestructura y servicio invocable.
(sin análogo MLDCAT-AP)	Extensión puente	Formatos de entrada/salida y condiciones de autenticación.
(sin análogo MLDCAT-AP)	Extensión puente	Salida gobernable de una ejecución autorizada.
(sin análogo MLDCAT-AP)	Extensión puente	Evidencia de integridad y auditoría orientada a cumplimiento.
(sin análogo MLDCAT-AP)	Extensión puente	Agregación de procedencia entre contextos de participante.
(sin análogo MLDCAT-AP)	Extensión puente	Condiciones reificadas de comparabilidad de evaluaciones.
(sin análogo MLDCAT-AP)	Extensión puente	Rol de participante en el intercambio dentro del dataspace.

Tabla 2. Matriz de reuso: familias de trabajo relacionado contra los cinco atributos operacionales que un perfil para modelos de IA gobernados debe cubrir. ● = cubierto nativamente; ○ = cubierto parcialmente mediante extensiones; — = no cubierto.

Familia / artefacto	Publicación	Política	Invocación	Trazabilidad	Evaluación
Model cards, factsheets, datasheets ^{6,7,19}	○	—	—	—	○
ML-Schema, MEX, OntoDM, DMOP ^{23–26}	—	—	—	○	●
AIO, ITO, gestión modelo-dataset ^{27–29}	—	—	—	○	●
MLDCAT-AP 3.0.0 ³⁵	●	○	—	●	●
DCAT 2 ²	●	—	○	—	—
ODRL 2.2 ⁴	—	●	—	—	—
PROV-O ³	—	—	—	●	○
DSP / EDC ^{37,39}	○	○	●	—	—
DAIMO (este trabajo)	●	●	●	●	●

está cubierta nativamente por al menos un vocabulario reutilizable. DAIMO se diseña, por tanto, como un perfil de integración que ata esas cinco capacidades a través de alineamientos axiomáticos verificados, sin duplicar lo que los vocabularios maduros ya resuelven.

Metodología y requisitos

DAIMO adopta la metodología *Linked Open Terms* (LOT)³² por dos razones. Primera, su orientación industrial: LOT prescribe un flujo ligero con entregables en cada una de sus cuatro fases (especificación de requisitos, implementación, publicación y mantenimiento). Segunda, LOT trata la reutilización como fase de primera clase,

no como subproducto, lo que encaja con la regla *reuse-first* que el problema impone. Las subsecciones siguientes describen el escenario ejecutivo que guió la elicitación de requisitos (§), las preguntas de competencia y los requisitos no funcionales resultantes (§), y las tres decisiones centrales de diseño que dieron forma a la ontología (§).

Escenario ejecutivo

La especificación de requisitos parte de un escenario ejecutivo anonimizado que posteriormente actúa como caso de estudio (§). Un Proveedor A publica un modelo de predicción geoespacial de riesgo en un catálogo federado gestionado por una Plataforma C. Un Consumidor municipal B necesita invocar el modelo para apoyar decisiones operativas tras un evento extremo, pero no puede hacerlo sin antes descubrirlo bajo una política compatible con su uso público, verificar que satisface un umbral mínimo de calidad sobre un contexto de evaluación común, negociar un acuerdo que autorice sus ejecuciones concretas e invocarlo a través de un endpoint autenticado. Un Evaluador científico D compara el modelo con una línea base sobre el mismo contexto de evaluación. Un Actor de cumplimiento E audita las ejecuciones y archiva la evidencia correspondiente. Los nombres, modelos, endpoints, métricas y fechas del escenario son sintéticos por diseño: el caso no se presenta como piloto desplegado, sino como demostrador reproducible de adecuación representacional.

Este escenario sintetiza las cinco tareas operativas que una ontología del puente catálogo-dataspaces debe soportar simultáneamente (publicación, descubrimiento, invocación, trazabilidad y evaluación) e involucra a cinco tipos de actor claramente diferenciados cuyas necesidades son complementarias pero no coinciden. Es verosímil: no depende de tecnología hipotética y se puede instanciar con los vocabularios reutilizados más las extensiones que DAIMO introduce.

Especificación de requisitos

Los requisitos se elicitaban desde tres fuentes complementarias: el escenario anterior, el dossier interno de MLDCAT-AP 3.0.0 y el análisis de las extensiones Eclipse EDC empleadas por INESData. A partir de ellas se formularon veintitrés preguntas de competencia, organizadas en cinco categorías por actor y etapa del ciclo de vida. La categoría R (registro y publicación, cinco preguntas) cubre las necesidades del proveedor al publicar el modelo como activo gobernado con identidad, licencia, política y contrato de invocación. La categoría D (descubrimiento y selección, cuatro preguntas) recoge las preguntas que un consumidor plantea para localizar un modelo adecuado por tarea, dominio, política y umbral de calidad. La categoría E (ejecución y auditabilidad, cinco preguntas) agrupa las preguntas que un operador y un actor de gobernanza se plantean para invocar servicios, trazar ejecuciones y reconstruir evidencia. La categoría V (evaluación y reproducibilidad, cinco preguntas) contiene las preguntas del evaluador para comparar resultados bajo condiciones homogéneas. Finalmente, la categoría G (puente de gobernanza, cuatro preguntas) cubre las preguntas multi-actor cuya respuesta exige exclusivamente semántica nativa de DAIMO y que, como se argumenta en §, no pueden

responderse con los vocabularios reutilizados aisladamente. La Tabla 3 resume la correspondencia actor-categoría; la Tabla 4 traza cada familia de requisitos hacia los compromisos de modelado que la satisfacen; la Tabla 5 normaliza los términos nucleares del dominio antes de introducir clases o propiedades; el conjunto completo de las veintitrés preguntas en lenguaje natural aparece en el Apéndice.

El flujo operacional de ejecución federada se interpreta en DAIMO como una cadena semántica auditable, no como una arquitectura de runtime. El descubrimiento se representa mediante ofertas gobernadas y CQs D1–D4; la negociación, mediante `odrl:Agreement` y `ExecutionAuthorization`; el despliegue, mediante `ModelDeployment`, `dcat:DataService` e `IOContract`; la inferencia, mediante `it6:Run` y `DerivedArtifact`; y el cierre operativo o *clearing*, mediante `AuditEvidence` y `CrossParticipantProvenanceRecord`. Esta lectura preserva la premisa de soberanía: DAIMO puede describir ejecuciones en el nodo del proveedor de datos, en un nodo neutral o en una infraestructura acordada, pero no prescribe si el modelo viaja al dato, el dato al modelo, o si la ejecución ocurre dentro de un mecanismo particular de aislamiento.

La enumeración de términos se trató como un artefacto intermedio entre las preguntas de competencia y la TBox. La Tabla 5 no añade semántica nueva: normaliza el vocabulario operativo que aparece en el escenario, decide si cada término se reutiliza o se especializa, y evita convertir temas generales en clases nativas sin necesidad. Las definiciones formales completas se mantienen en la ontología mediante `skos:definition`, `skos:example`, dominios, rangos y shapes.

Once de las veintitrés preguntas exigen razonamiento explícito (rutas de subclase, cadenas de subpropiedad o agregación SPARQL), mientras que las doce restantes son consultas directas sobre datos DAIMO-nativos. Esta proporción es deliberada: un perfil que se apoyara exclusivamente en consultas directas no ejercitaría la parte del artefacto que hace valiosa la semántica OWL, y un perfil que exigiera razonamiento para toda pregunta sería impracticable en operación sobre endpoints reales. Los requisitos no funcionales complementan los funcionales y fijan: perfil OWL 2 DL (para mantener decidibilidad y compatibilidad con HermiT), licencia CC-BY 4.0 para la ontología y Apache 2.0 para los scripts de validación, URIs persistentes bajo <https://w3id.org/pionera/daimo> con negociación de contenido hacia las cinco serializaciones estándar, validación SHACL accesible y, sobre todos ellos, un compromiso explícito de *reuse-first*: cada término introducido por DAIMO debe justificar en la documentación por qué DCAT, DCAT-AP, MLDCAT-AP, ODRL, PROV-O, DSP o la extensión EDC no lo cubren.

Decisiones centrales de diseño

Tres decisiones estructurales se tomaron en la fase de conceptualización y condicionan el resto del diseño.

Decisión 1: reutilización preferente. DAIMO reutiliza directamente las clases de MLDCAT-AP para todo aquello que MLDCAT-AP ya modela con precisión.

Tabla 3. Actores involucrados en el escenario ejecutivo y categorías de preguntas de competencia derivadas.

Actor	Categoría	Necesidad	CQs
Proveedor de modelos	R	Publicar un modelo como activo gobernado con identidad, licencia, política y contrato de invocación	CQ-R1–CQ-R5
Consumidor de modelos	D	Descubrir modelos por tarea, dominio, política y umbral de calidad	CQ-D1–CQ-D4
Operador de plataforma	E	Invocar servicios, trazar ejecuciones y reconstruir evidencia operativa	CQ-E1–CQ-E5
Evaluador	V	Comparar resultados en un contexto compartido e inspeccionar reproducibilidad	CQ-V1–CQ-V5
Gobernanza (multi-actor)	G	Puentear oferta, despliegue, autorización y procedencia entre participantes	CQ-G1–CQ-G4

Tabla 4. Trazabilidad desde requisito operacional hacia compromiso de modelado. La tabla explicita por qué existe cada grupo de modelado y qué artefacto de evaluación lo ejerce.

Familia de requisito	Pregunta que el perfil debe responder	Compromiso DAIMO principal	Evidencia
Publicación	¿Este modelo es un activo gobernado en un contexto de catálogo concreto?	Registro de oferta que vincula modelo, proveedor, contexto de catálogo y política aplicable.	CQs R1–R5, G1
Descubrimiento	¿Puede un consumidor filtrar modelos por tarea, dominio, política y umbral de evaluación?	Semántica de modelo, tarea y evaluación de MLDCAT-AP más enlaces a política y contexto de evaluación.	CQs D1–D4
Invocación	¿Qué servicio, método de autenticación y contrato E/S aplican a un modelo seleccionado?	Despliegue de modelo conectado a servicios DCAT y contratos E/S, permitiendo varios endpoints por modelo alojado.	CQs E1, G2
Autorización	¿Qué acuerdo autorizó una ejecución concreta por un participante concreto?	Acuerdo negociado enlazado al agente beneficiario y a la ejecución que cubre.	CQs E2, G3; INV-2, INV-4
Auditabilidad	¿Puede un actor de gobernanza reconstruir artefactos, evidencias y procedencia entre contextos?	Artefacto derivado, evidencia de auditoría y bundle de procedencia entre participantes.	CQs E4, E5, G4; INV-1
Evaluación	¿Son comparables las mediciones bajo la misma tarea, versión de dataset, protocolo y semilla?	Contexto compartido de evaluación que fija tarea, dataset, versión, protocolo y semilla.	CQs V1–V5

`it6:MachineLearningModel` es la clase del modelo; `it6:ComputerInfrastructure` describe el entorno de ejecución; `it6:Run`, `it6:Flow` e `it6:Evaluation` capturan respectivamente la ejecución concreta, el *pipeline* asociado y la medición resultante; `dcat:Dataset`, `dcat:Distribution` y `dcat:DataService` publican el activo; `odrl:Offer` y `odrl:Agreement` expresan la política y el acuerdo. Introducir equivalentes DAIMO-nativos de cualquiera de estas clases violaría el principio de compromiso ontológico mínimo de Gruber y fracturaría el ecosistema DCAT-AP sin ganancia; una clase DAIMO-nativa sólo se justifica cuando la unión de los vocabularios reutilizados deja un hueco concreto que impide responder a al menos una pregunta de competencia.

Esta decisión ordena el modelo conceptual sin convertir cada tema en una clase nueva. El modelo de IA se mantiene como `it6:MachineLearningModel`; los recursos de datos como `dcat:Dataset` y `dcat:Distribution`; la infraestructura como `it6:ComputerInfrastructure`; las métricas como `it6:Evaluation` y `it6:EvaluationMeasure`; y la gobernanza como combinación de políticas ODRL, autorizaciones DAIMO, roles de participante y evidencia

auditable. El linaje no se reifica como una clase monolítica, sino como una cadena consultable que conecta datos, ejecución, despliegue, artefactos derivados y bundles PROV. Por la misma razón, candidatos habituales como `AIModel`, `DataSet`, `Algorithm`, `ModelVersion`, `hasOwner`, `hasLicense`, `frameworks` o hiperparámetros se reutilizan desde MLDCAT-AP, DCAT, ODRL, PROV-O o ML-Schema, o quedan como metadatos de perfiles externos si ninguna pregunta de competencia del núcleo los exige. Así, DAIMO permanece desacoplada de frameworks concretos como PyTorch o TensorFlow y evita importar modelos IDS o Gaia-X como dependencias del núcleo; esos ecosistemas quedan como contextos de adopción interoperables mediante DCAT, ODRL, PROV-O, DSP y los alineamientos del perfil.

Decisión 2: separación en tres capas complementarias. La arquitectura distingue tres capas que DAIMO conecta pero no sustituye. La capa A es el plano del espacio de datos: DSP con las extensiones EDC puntuales que DAIMO referencia. La capa B es el plano de gobernanza de plataforma: INESData (o una plataforma análoga) con sus mecanismos de registro federado y políticas operativas. La capa C es el plano del activo de IA/ML: MLDCAT-AP y los vocabularios que lo soportan. DAIMO es el perfil de

Tabla 5. Glosario inicial de términos nucleares del dominio y su realización ontológica.

Término del dominio	Sentido operacional	Realización ontológica
Modelo de IA	Artefacto entrenado o empaquetado que puede publicarse, evaluarse e invocarse.	Reutilizado mediante la clase de modelo de MLDCAT-AP; DAIMO no duplica la clase del modelo.
Oferta gobernada	Acto de registrar un modelo como activo disponible bajo política en un catálogo federado.	AIAssetOffering enlaza modelo, agente y odrl:Offer; se alinea con dcat:CatalogRecord.
Participante	Agente jurídico u organizativo que actúa dentro del espacio de datos.	foaf:Agent para el agente; ParticipantRole para el rol contextual.
Rol de participante	Papel asumido respecto al activo: proveedor, consumidor, operador, evaluador o gobernanza.	Cinco subclases no disjuntas de ParticipantRole, porque un agente puede asumir varios roles.
Contexto de participante	Ámbito administrativo donde se publica, negocia, opera o audita el intercambio.	Reutilizado como edc:ParticipantContext; DAIMO lo usa para acotar roles y procedencia.
Política y acuerdo	Condiciones de uso de la oferta y resultado de una negociación de acceso.	Reutilización de odrl:Offer y odrl:Agreement; DAIMO especializa sólo la autorización de ejecución.
Despliegue de modelo	Instancia alojada que sirve un modelo mediante uno o más servicios.	ModelDeployment conecta modelo, infraestructura, servicio DCAT y contrato E/S.
Servicio o endpoint	Interfaz accesible para invocar un despliegue.	Reutilizado como dcat:DataService; DAIMO añade el enlace desde el despliegue.
Contrato E/S	Formato de entrada, formato de salida, autenticación y esquemas opcionales.	IOContract con inputFormat, outputFormat, authMethod y esquemas opcionales.
Ejecución autorizada	Invocación concreta cubierta por un acuerdo válido.	it6:Run para la ejecución y ExecutionAuthorization para el acuerdo que la autoriza.
Artefacto derivado	Resultado gobernable producido por una ejecución.	DerivedArtifact, enlazado a la ejecución y a la autorización bajo la que se produjo.
Evidencia de auditoría	Registro verificable que respalda una actividad ejecutada.	AuditEvidence con actividad, firmante, marca temporal y checksum SPDIX.
Procedencia cruzada	Narrativa de procedencia que cruza al menos dos contextos de participante.	Clase DAIMO de procedencia cruzada como bundle PROV con contextos explícitos.
Contexto de evaluación	Condiciones que hacen comparables las métricas de dos modelos.	SharedEvaluationContext fija tarea, dataset, versión, protocolo, semilla y flow.
Evaluación y métrica	Medición de desempeño usada para comparar modelos.	Reutilización de it6:Evaluation; DAIMO enlaza la evaluación al contexto compartido.

integración que conecta A, B y C; no es una cuarta capa paralela que replique funcionalidad, sino un puente que hace consultable la articulación entre las tres.

Decisión 3: una clase nativa por hueco, no por tema. Dada la reutilización preferente, cada clase DAIMO-nativa debe justificar su existencia por un hueco concreto en la unión de los vocabularios reutilizados y por al menos una pregunta de competencia que no puede responderse sin ella. El criterio se aplica con rigor y produce nueve clases de primer nivel (AIAssetOffering, ParticipantRole, ModelDeployment, IOContract, ExecutionAuthorization, DerivedArtifact, CrossParticipantProvenanceRecord, AuditEvidence y SharedEvaluationContext) más cinco subclases de rol. Cualquier clase que no cumpla simultáneamente ambas condiciones queda descartada.

La ontología DAIMO

Arquitectura modular y métricas

DAIMO modela el ciclo de vida operacional de un activo de modelo de IA dentro de un espacio de datos gobernado: publicación como registro de catálogo, descubrimiento sensible a política y a calidad, invocación controlada mediante servicios tipados, trazabilidad de ejecuciones individuales y evaluación contextualizada y reproducible. El alcance se restringe deliberadamente a esta cadena operacional. Las responsabilidades ortogonales, como la modelización detallada del *pipeline* de entrenamiento, la semántica fina de políticas de privacidad o la teoría formal de la derivación PROV, ya están cubiertas por vocabularios consolidados cuya duplicación fragmentaría el ecosistema. DAIMO aporta, en cambio, los constructos necesarios para que esos vocabularios se articulen coherentemente en el caso

concreto de un modelo de IA que circula entre participantes autónomos.

La ontología se estructura en tres módulos complementarios, distribuidos en archivos Turtle separados para que un consumidor pueda cargar únicamente los que necesite. El **módulo núcleo** (`daimo-core.ttl`) contiene las catorce clases y las veintinueve propiedades de objeto más ocho propiedades de datatype DAIMO-nativas, los axiomas globales de disyunción y las caracterizaciones sobre propiedades (funcionales, asimétricas, inversas) que capturan las invariantes de tipo a nivel TBox. El **módulo de alineamientos** (`alignment.ttl`) encapsula ocho declaraciones `rdfs:subClassOf` externas y cinco declaraciones `rdfs:subPropertyOf` externas que conectan DAIMO con los vocabularios reutilizados, más las declaraciones mínimas de los términos externos que DAIMO referencia, cada una con un `rdfs:seeAlso` a la sección correspondiente de la especificación fuente; en conjunto, el paquete contiene seis subpropiedades DAIMO hacia vocabularios externos porque `hasOffering` $\sqsubseteq$ `foaf:isPrimaryTopicOf` se declara en el núcleo como inversa de `offersModel`. El **módulo SHACL** (`daimo-shapes.ttl`) aporta las dieciocho `sh:NodeShape` del perfil: nueve estructurales, tres de conformidad sobre clases reutilizadas y seis invariantes entre clases mediante `sh:SPARQLConstraint`. El módulo se declara como `owl:Ontology` independiente con su propia `owl:versionIRI`, de modo que la capa de validación se libera y referencia como un artefacto con ciclo de vida propio.

La separación en tres módulos no es accidental. El núcleo puede ser consumido por una aplicación que sólo necesite las primitivas DAIMO; el módulo de alineamientos se carga cuando se quiere que un razonador infiera las relaciones entre DAIMO y los vocabularios reutilizados; el módulo SHACL se carga cuando se quiere validar un grafo concreto contra

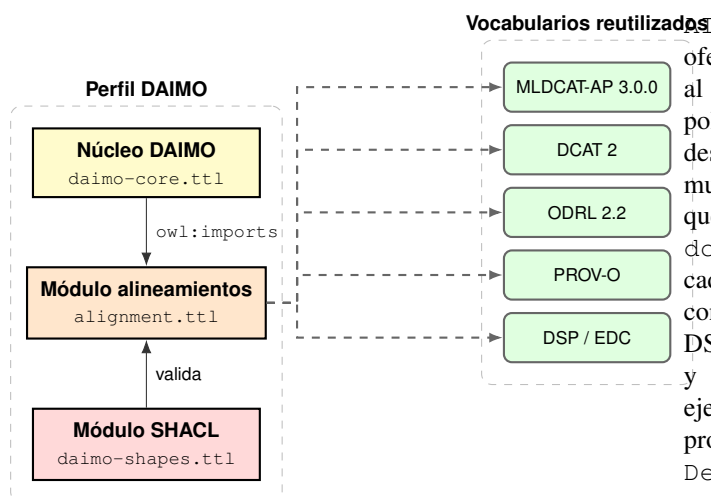


Figura 1. Arquitectura modular de DAIMO y vocabularios reutilizados. Las cajas amarilla, naranja y roja representan los tres módulos de DAIMO (núcleo, alineamientos, shapes); las verdes, los vocabularios reutilizados. Las flechas discontinuas desde el módulo de alineamientos portan las declaraciones `rdfs:subClassOf` y `rdfs:subPropertyOf`. El núcleo importa alineamientos para inferencia; el módulo SHACL valida datos contra el TBox combinado.

el perfil. La Figura 1 ilustra la arquitectura modular y las relaciones con los vocabularios reutilizados.

Métricas del artefacto. Antes de recorrer las clases individuales que pueblan los tres módulos de §, reportamos el tamaño y la expresividad del artefacto globalmente. La Tabla 7 extrae las métricas directamente de los archivos Turtle y de los reportes producidos por la suite de validación (§). Los números permiten a un lector interesado estimar el peso del artefacto antes de consumirlo y compararlo con ontologías publicadas previamente en la misma familia.

Clases nativas y alineamientos

Las nueve clases nativas de primer nivel, más las cinco subclases de `ParticipantRole`, constituyen el conjunto mínimo que las tres decisiones centrales de diseño producen. La Tabla 9 sintetiza cada una junto con su alineamiento axiomático a los vocabularios reutilizados; las subsecciones siguientes detallan las de mayor carga semántica. La Figura 2 muestra la vista general de clases y propiedades nativas.

Las dos clases sin alineamiento externo, `IOContract` y `SharedEvaluationContext`, carecen de él deliberadamente. `IOContract` no es un `dcat:DataService` (un servicio es un recurso accesible, una URL, un endpoint, mientras que el contrato describe expectativas sintácticas y de autenticación sobre el intercambio) ni un `dct:Standard` (el contrato enumera obligaciones concretas, no refiere a una especificación externa). `SharedEvaluationContext` es una reificación metodológica, no una tarea: captura la combinación exacta bajo la que dos evaluaciones son comparables, no la tarea abstracta que dichas evaluaciones instancian. Alinearla a `owl:Thing` o `prov:Entity` añadiría ruido sin aportar semántica.

En el escenario ejecutivo de §, el proveedor publica sus modelos mediante tres instancias de

`AssetOffering` en el catálogo federado; cada oferta enlaza al modelo con `daimo:offersModel`, al proveedor con `daimo:offeredBy` y a la política ODRL con `daimo:hasOfferPolicy`. El despliegue del modelo seleccionado por el consumidor municipal se modela como una `ModelDeployment` que, mediante `daimo:exposedAs`, apunta a dos `dcat:DataServices` paralelos (REST y gRPC), cada uno con su `IOContract`. La autorización que el consumidor obtiene del proveedor tras la negociación DSP se reifica como `ExecutionAuthorization` y vincula, vía `daimo:authorizesRun`, las ejecuciones concretas que se permiten. La predicción producida por cada ejecución se reifica como `DerivedArtifact` con su `AuditEvidence` asociada, y el `CrossParticipantProvenanceRecord` agrega la cadena completa a través de los `edc:ParticipantContexts` del proveedor, el consumidor y la plataforma federada.

Registro y participación. `AIAssetOffering` y `ParticipantRole` resuelven conjuntamente el hueco del lado de publicación. `MLDCAT-AP` describe el modelo, y `DCAT` registra metadatos de catálogo, pero ninguno expresa el acto gobernado de poner un modelo a disposición de otro participante de un espacio de datos. La oferta se modela por tanto como el registro de catálogo de un modelo en un dataspace concreto, acotado por política, proveedor, modelo objetivo y contexto de participante. `ParticipantRole`, con cinco especializaciones no disjuntas, registra el rol que un agente jurídico asume respecto de un activo y un contexto de datos. El grupo responde a CQ-R1, CQ-R2, CQ-R5 y CQ-G1.

Plano de ejecución. `ModelDeployment` e `IOContract` cubren el hueco de invocación. `DCAT` proporciona servicios visibles desde catálogo, pero no especifica qué modelo aloja un servicio ni qué expectativas sintácticas y de autenticación debe satisfacer el consumidor para invocarlo. Un despliegue representa la instancia alojada o en ejecución de un modelo sobre infraestructura de cómputo. Puede exponerse mediante uno o varios servicios, y cada servicio puede declarar un contrato de invocación con formato de entrada/salida, método de autenticación y referencias opcionales a esquemas. La invocación no se modela como una propiedad directa modelo-consumidor: se reconstruye recorriendo oferta, despliegue, servicio, contrato E/S, autorización y `it6:Run`, lo que evita introducir relaciones redundantes como `isInvokedBy`. El perfil registra dónde y bajo qué contrato se puede invocar el modelo; no modela la orquestación de contenedores, el motor de enforcement ni el mecanismo TEE, que deben declararse como metadatos de infraestructura o perfiles específicos cuando una política los exige. El grupo responde a CQ-E1, CQ-E3 y CQ-G2.

Autorización y derivación. `ExecutionAuthorization` y `DerivedArtifact` cubren la cadena autorización-salida. ODRL aporta acuerdos vinculantes, y PROV-O relaciones generales de generación y derivación, pero ninguno registra el acto específico de autorizar una ejecución concreta de aprendizaje automático ni el artefacto gobernable que esa ejecución produce. La autorización se modela como el acuerdo negociado que vincula permisos con ejecuciones;

Tabla 6. Perfil técnico de implementación de DAIMO. La tabla traduce requisitos habituales de implementación a artefactos existentes del paquete, sin introducir módulos ni clases adicionales.

Pilar	Implementación en DAIMO	Decisión de alcance
Lenguaje, serialización e IRI	Núcleo OWL 2 DL en Turtle; serializaciones RDF/XML, JSON-LD y N-Triples preparadas para publicación; namespace canónico https://w3id.org/pionera/daimo .	El despliegue <code>w3id</code> y el DOI son pasos FAIR finales; la reproducibilidad local no depende de ellos.
Vistas conceptuales de implementación	Metadatos de modelo: MLDCAT-AP; datos y catálogo: DCAT; ejecución e infraestructura: MLDCAT-AP más <code>ModelDeployment/IOContract</code> ; gobernanza y confianza: ODRL, autorizaciones, evaluación contextualizada y evidencia.	Estas son vistas funcionales, no módulos ontológicos nuevos como <code>ModelMetadata</code> o <code>GovernanceTrust</code> .
Relaciones críticas	Se reutiliza <code>it6:trainedOn</code> ; los formatos y esquemas se expresan en <code>IOContract</code> ; el despliegue se conecta al endpoint mediante <code>exposedAs</code> y <code>dcat:DataService</code> ; la conformidad normativa puede expresarse con <code>dct:conformsTo</code> o perfiles ODRL/SHACL específicos.	No se añaden propiedades duplicadas como <code>trainedOn</code> , <code>deployedIn</code> o <code>compliesWith</code> si ya existe una ruta estándar o perfilable.
Validación y razonamiento	SHACL estructural, SHACL sobre clases reutilizadas, seis invariantes SHACL-SPARQL, Hermit, OWL-RL, OOPS!, pruebas negativas y 23 consultas SPARQL.	No se exige una métrica universal como <i>accuracy</i> ; las métricas obligatorias dependen del contexto de evaluación y de la CQ ejercitada.
Almacenamiento y acceso	El paquete RDF puede cargarse en triplestores estándar y exponerse por SPARQL; las consultas publicadas en <code>daimo/queries/queries.md</code> demuestran el patrón de acceso.	El artículo no afirma un endpoint productivo GraphDB/Fuseki; sólo compatibilidad técnica con ese tipo de despliegue.

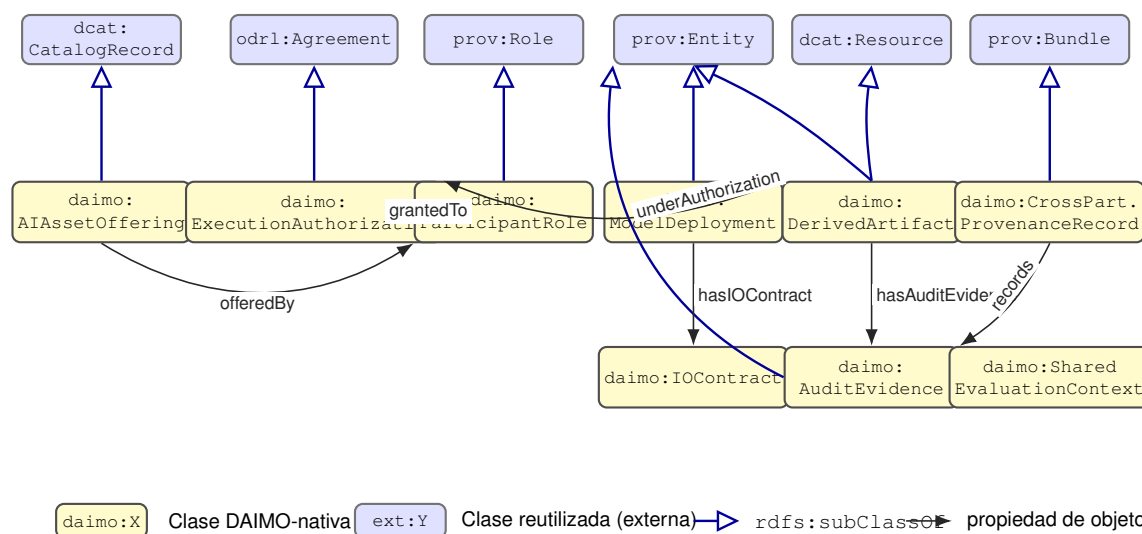


Figura 2. Vista general de las clases nativas de DAIMO (amarillo) con sus alineamientos axiomáticos hacia los vocabularios reutilizados (azul) mediante `rdfs:subClassOf` (flechas triangulares huecas), y las principales propiedades nativas que articulan el perfil (flechas etiquetadas). `DerivedArtifact` tiene dos padres externos, `prov:Entity` y `dcat:Resource`; `AuditEvidence` aparece en la fila inferior pero es también una `prov:Entity`. Las clases `IOContract` y `SharedEvaluationContext` carecen de alineamiento externo por diseño. Las propiedades cuyos destinos son clases reutilizadas (`offersModel` hacia `it6:MachineLearningModel`, `authorizesRun` hacia `it6:Run`) aparecen en la Tabla 9 y se omiten aquí para no sobrecargar el diagrama.

el artefacto derivado se modela como la salida gobernada de tal ejecución. La invariante INV-1 fuerza la consistencia de esta cadena. El grupo responde a CQ-E2, CQ-E5, CQ-G3 y CQ-G4.

Auditoría y agregación de procedencia. `AuditEvidence` y `CrossParticipantProvenanceRecord` cubren la auditabilidad a través de fronteras administrativas. `SPDX` proporciona checksums, y `PROV-O` bundles de procedencia, pero ninguno atiende por sí solo la evidencia orientada a cumplimiento que cruza contextos de participante. La evidencia de auditoría combina un valor de integridad, un firmante y una marca temporal; el registro de procedencia cruzada agrega actividades y entidades relevantes entre contextos de participante, produciendo una narrativa de auditoría que un actor de gobernanza puede recorrer extremo a extremo. El grupo responde a CQ-E4 y CQ-G4.

Contexto de evaluación. `SharedEvaluationContext` cubre el hueco de comparabilidad que las evaluaciones MLDCAT-AP dejan abierto. MLDCAT-AP registra una medición asociada a un modelo; no reifica la combinación de tarea, versión de dataset, protocolo de evaluación y semilla aleatoria que hace comparables dos mediciones. Las métricas se modelan como recursos vinculados a una evaluación y a un contexto, no como atributos fijos del modelo: la misma versión de modelo puede exhibir precisión, sesgo, latencia o coste computacional distintos bajo datasets, protocolos o infraestructuras distintas. Dos evaluaciones que comparten este contexto son comparables por construcción; evaluaciones que difieren en cualquiera de esas coordenadas no lo son. Esta comparabilidad es operacional y mínima: presupone que la métrica referenciada tiene la misma definición, unidad y procedimiento de cálculo, aspectos que pueden declararse en perfiles de métricas o protocolos

Tabla 7. Métricas del artefacto DAIMO verificadas contra los archivos del paquete.

Dimensión	Valor
Perfil lógico declarado	OWL 2 DL
Clases nativas de primer nivel	9
Subclases nativas (ParticipantRole)	5
Total clases DAIMO-nativas	14
Propiedades de objeto (owl:ObjectProperty)	29
Propiedades de datatype (owl:DatatypeProperty)	8
Propiedades funcionales (owl:FunctionalProperty)	27
Propiedades asimétricas (owl:AsymmetricProperty)	5
Pares inversos explícitos (owl:inverseOf)	5
rdfs:subClassOf a externos	8
rdfs:subPropertyOf a externos	6
Axioma de disyunción nombrado	1
sh:NodeShape totales	18
de completitud (una por clase DAIMO)	9
de conformidad sobre reutilizadas	3
invariantes	6
sh:SPARQLConstraint	
Preguntas de competencia	23
Pitfalls OOPS! (Crítico/Importante/Menor)	0 / 0 / 2

Tabla 8. Desglose estilo OntoMetrics extraído de los módulos fuente DAIMO. Los conteos se restringen a sujetos del namespace DAIMO para evitar inflar la métrica con declaraciones externas usadas sólo para alineamiento.

Tipo de axioma/declaración	Conteo
owl:Class	14
owl:ObjectProperty	29
owl:DatatypeProperty	8
rdfs:subClassOf	13
rdfs:subPropertyOf	6
rdfs:domain	37
rdfs:range	37
owl:FunctionalProperty	27
owl:AsymmetricProperty	5
owl:inverseOf	5
owl:AllDisjointClasses	1
owl:equivalentClass	0
owl:disjointWith explícitos	0

cuando el espacio de datos los necesite. La clase responde a CQ-V1, CQ-V2 y CQ-V3.

Defensa de los alineamientos de mayor carga. De los ocho alineamientos `rdfs:subClassOf` hacia vocabularios externos, dos llevan significativamente más carga semántica que el resto y merecen defensa explícita porque las contra-lecturas son plausibles y un lector familiarizado con DCAT y ODRL las planteará inmediatamente.

$AIAssetOffering \sqsubseteq dcat:CatalogRecord$, $no\ dcat:Dataset$. DCAT distingue con cuidado entre el recurso ofrecido (el `dcat:Dataset`, que es la cosa sobre la que hay interés) y el registro que describe dicho recurso

en un catálogo concreto (el `dcat:CatalogRecord`, que es el artefacto de metadatos). DAIMO alinea `AIAssetOffering` a `dcat:CatalogRecord` porque la oferta es conceptualmente el registro de catalogación del modelo en un espacio de datos concreto, con una política y un conjunto de metadatos específicos de ese contexto, no el modelo en sí —que se reutiliza directamente como `it6:MachineLearningModel`. La contra-lectura que subsumiría `AIAssetOffering` a `dcat:Dataset` conlugaría dos cosas distintas: el modelo (que puede aparecer en varios catálogos con condiciones distintas) y su oferta en un catálogo específico (que es una y con una política propia). La distinción es operacionalmente relevante: una misma `it6:MachineLearningModel` puede ser referenciada por varias `AIAssetOfferings` con políticas distintas, y la propiedad `daimo:offersModel`, alineada a `foaf:primaryTopic`, captura que la oferta trata sobre el modelo sin identificarse con él.

$ExecutionAuthorization \sqsubseteq odrl:Agreement$, $no\ prov:wasDerivedFrom\ odrl:Agreement$. La autorización de ejecución es el resultado específico de una negociación DSP que vincula permisos ODRL concretos a invocaciones concretas, y ODRL modela precisamente este tipo de instrumento vinculante como `odrl:Agreement` —el equivalente a un contrato suscrito entre un asignador y un asignatario, con permisos y restricciones efectivas. La contra-lectura que relacionaría la autorización con el acuerdo mediante `prov:wasDerivedFrom` trataría la autorización como un artefacto distinto del acuerdo, introduciendo una distinción que ODRL no necesita: un `Agreement` ya es la entidad vinculante. La subsunción aprovecha toda la semántica ODRL sobre permisos, prohibiciones y obligaciones sin redefinirla, y permite que herramientas ODRL existentes traten las autorizaciones DAIMO como acuerdos estándar. Lo que DAIMO añade sobre `odrl:Agreement` es la propiedad asimétrica `daimo:authorizesRun`, que vincula el acuerdo a las ejecuciones `it6:Run` que cubre: una relación que ODRL no prescribe pero que el puente al plano PROV requiere.

Los cinco alineamientos restantes se justifican a continuación, ordenados desde el más discutible hasta el menos problemático.

$ParticipantRole \sqsubseteq prov:Role$. PROV-O acota `Role` a una asociación cualificada dentro de una actividad, mientras que el rol DAIMO se acota a un contexto de dataspace y normalmente sobrevive a una ejecución concreta. La subsunción es defendible sólo en sentido *is-a-kind-of*: un rol DAIMO es un rol PROV cuando cualifica una actividad específica, y la diferencia de alcance se captura mediante la propiedad que lo vincula a un `edc:ParticipantContext`. La alternativa menos comprometida sería `skos:related` con `prov:Role`; se descarta porque impediría que los razonadores clasifiquen roles DAIMO como roles PROV cuando efectivamente cualifican una asociación. Se acepta la subsunción con la condición documentada de que el contexto DAIMO añade alcance administrativo, no contradicción semántica.

$ModelDeployment \sqsubseteq prov:Entity$. Un despliegue es un artefacto persistente sobre el que ocurren actividades (carga del modelo, inferencia, logging), no una

Tabla 9. Clases nativas de DAIMO, alineamientos axiomáticos y preguntas de competencia que responden.

Clase	Alineamiento	Papel	CQs
daimo:AIAsset Offering	dcat:CatalogRecord	Registro de catálogo que publica un modelo como oferta gobernada en un dataspace; porta política ODRL mediante daimo:hasOfferPolicy.	R1, R2, R5, G1
daimo:ParticipantRole (+5 subclases no disjuntas)	prov:Role	Rol asumido por un participante respecto a un activo de IA dentro de un contexto de dataspace.	R5
daimo:Model Deployment	prov:Entity	Instancia en ejecución o alojada del modelo, expuesta como dcat:DataService sobre una it6:ComputerInfrastructure.	E1, E3, G2
daimo:IOContract	(sin alineamiento)	Contrato mínimo accionable: formatos de E/S, método de autenticación, esquemas opcionales.	R4, E1, G2
daimo:Execution Authorization	odrl:Agreement	Acuerdo ODRL derivado de una negociación DSP que autoriza ejecuciones concretas vía daimo:authorizesRun.	E2, E5, G3
daimo:DerivedArtifact	prov:Entity, dcat:Resource	Salida gobernada y describable en catálogo producida por una ejecución bajo autorización.	E5, G4
daimo:CrossParticipant ProvenanceRecord	prov:Bundle	Agregación de actividades y entidades a través de contextos de participante para formar una narrativa de auditoría.	G4
daimo:AuditEvidence	prov:Entity	Registro de evidencia de cumplimiento con spdx:Checksum estructurado, firmante y marca temporal.	E4
daimo:SharedEvaluation Context	(sin alineamiento)	Reificación de las condiciones de comparabilidad: tarea, dataset, versión, protocolo y semilla aleatoria.	V1, V2, V3

actividad en sí. Modelarlo como `prov:Activity` forzaría marcas `startedAtTime/endedAtTime`, adecuadas para una ejecución pero no para una instancia alojada de larga duración. Alinear el despliegue con `prov:Entity` preserva la intuición de que poner un modelo en una infraestructura produce una entidad gobernable sobre la que luego se ejecutan actividades.

$DerivedArtifact \sqsubseteq prov:Entity, dcat:Resource$. La doble subsunción refleja dos roles complementarios: el artefacto derivado es una entidad de procedencia producida por una ejecución y, al mismo tiempo, un recurso catalogable que un actor de gobernanza puede describir, listar o distribuir. `prov:Entity` y `dcat:Resource` son compatibles; DCAT no introduce compromiso alguno que contradiga PROV. El cierre OWL-RL del demostrador confirma que esta doble herencia no produce tipados no intencionados: la verificación de implicación inspecciona todas las superclases inferidas de `DerivedArtifact` y no emite advertencias.

$CrossParticipantProvenanceRecord \sqsubseteq prov:Bundle$. PROV-O ofrece dos constructos próximos: `prov:Bundle`, un conjunto nombrado de descripciones de procedencia con atribución propia, y `prov:Collection`, un conjunto de entidades sin metasemántica de procedencia. El registro DAIMO corresponde al primero: agrupa descripciones de procedencia a través de varios `edc:ParticipantContexts` y puede tener su propia atribución, fecha y evidencia. Alinear con `prov:Collection` perdería esa capa meta-provenance; alinear con ambos introduciría redundancia.

$AuditEvidence \sqsubseteq prov:Entity$. La evidencia es un artefacto producido (hash, firmante y marca temporal), no el acto de producirlo. La actividad de auditar se modela separadamente como una `prov:Activity` asociada al actor de cumplimiento; `AuditEvidence` es lo que esa actividad genera. Alinear la clase con `prov:Activity` conlugaría actividad y resultado, y debilitaría la distinción entidad-actividad sobre la que PROV se apoya.

Fundamentos axiomáticos

Una taxonomía por sí sola no fija el significado: sólo nombra los tipos. Detrás de la taxonomía de DAIMO hay una infraestructura axiomática que fija qué es cada clase, qué no es y cómo deben comportarse sus instancias. Tres estratos complementarios componen esta infraestructura: axiomas OWL sobre las clases y propiedades nativas, restricciones SHACL nodales sobre las instancias e invariantes SHACL-SPARQL entre clases. Los párrafos siguientes describen el primer estrato; los estratos SHACL aparecen en § como parte de la validación.

En el estrato OWL la disyunción entre los nueve tipos nativos de primer nivel se declara mediante el axioma nombrado `daimo:TopLevelKindsDisjointness`, instancia de `owl:AllDisjointClasses`. Convertir la disyunción en una entidad nombrada, en lugar de emitir pares anónimos `owl:disjointWith`, persigue tres beneficios. Primero, la disyunción puede referenciarse en documentación y reportes de validación. Segundo, un razonador puede citarla como causa específica cuando detecta una contradicción, facilitando el diagnóstico. Tercero, la disyunción entre las cinco subclases de `ParticipantRole` queda deliberadamente fuera del axioma: un mismo agente jurídico puede asumir

varios roles simultáneamente (por ejemplo, proveedor y operador) sin generar inconsistencia lógica, reflejando la realidad de que los roles son propiedades contextuales del agente y no clases mutuamente exclusivas.

Al núcleo de disyunciones se añaden caracterizaciones sobre las propiedades DAIMO. Veintisiete propiedades se declaran funcionales (`owl:FunctionalProperty`) sobre aquellas cuyo dominio es una clase con *shape* nodal, de modo que la cardinalidad máxima queda axiomatizada simultáneamente en la capa lógica y en la capa SHACL correspondiente. Cinco propiedades clave (`offersModel`, `deploysModel`, `authorizesRun`, `derivedFromRun` y `evidenceOf`) se marcan como asimétricas, capturando formalmente que un despliegue no es el modelo que despliega, que una autorización no coincide con la ejecución que autoriza, y así sucesivamente. Cinco pares inversos explícitos (`authorizedBy` ↔ `authorizesRun`, `hasDeployment` ↔ `deploysModel`, `hasDerivedArtifact` ↔ `derivedFromRun`, `hasAuditEvidence` ↔ `evidenceOf`, `hasOffering` ↔ `offersModel`) habilitan consultas inversas nativas sin obligar al publicador del grafo a materializar ambos sentidos.

El estrato SHACL se divide a su vez en dos familias. En primer lugar, cada clase DAIMO-nativa lleva asociada una *shape* nodal de completitud que prescribe qué propiedades son obligatorias en una instancia bien formada; estas nueve *shapes* son la garantía sintáctica de que un publicador ha suministrado todo lo necesario para que las invariantes semánticas operen sobre el grafo. En segundo lugar, tres *shapes* de conformidad (`OfferInDAIMOShape`, `MachineLearningModelInDAIMOShape` y `RunInDAIMOShape`) establecen qué propiedades de clases reutilizadas son obligatorias cuando un grafo declara adherirse al perfil DAIMO. La distinción entre ambas familias es deliberada: DAIMO no sobrescribe la ontología subyacente, porque eso fragmentaría el ecosistema, sino que declara su compromiso con un subconjunto concreto de propiedades sobre las clases externas, haciendo explícito el criterio de adhesión sin modificar las definiciones originales.

Por encima de las *shapes* se sitúan las seis invariantes SHACL-SPARQL entre clases (INV-1 a INV-6) que se examinan en detalle en §. Son el estrato que distingue a DAIMO de un simple perfil descriptivo: codifican reglas cuyo sentido no puede expresarse como restricción local a una sola *shape* porque involucran la coherencia entre varias clases simultáneamente. El Listado 1 muestra cómo se codifican los axiomas OWL para el caso concreto de `ExecutionAuthorization`, combinando la subclase con `odrl:Agreement`, la asimetría de `authorizesRun`, la funcionalidad de su inversa y la pertenencia al axioma de disyunción nombrado.

```
daimo:ExecutionAuthorization a owl:Class ;
  rdfs:subClassOf odrl:Agreement ;
  skos:definition "An ODRL agreement produced by
    a DSP contract
    negotiation that binds specific permissions
    to run invocations..."@en .

daimo:authorizesRun a owl:ObjectProperty , owl:
  AsymmetricProperty ;
```

```
rdfs:domain daimo:ExecutionAuthorization ;
rdfs:range it6:Run .

daimo:authorizedBy a owl:ObjectProperty , owl:
  FunctionalProperty ;
owl:inverseOf daimo:authorizesRun ;
rdfs:domain it6:Run ;
rdfs:range daimo:ExecutionAuthorization .

daimo:TopLevelKindsDisjointness a owl:
  AllDisjointClasses ;
owl:members ( daimo:AIAssetOffering
  daimo:ExecutionAuthorization
  daimo:ModelDeployment
  daimo:DerivedArtifact
  daimo:IOContract
  daimo:AuditEvidence
  daimo:
    CrossParticipantProvenanceRecord
  daimo:SharedEvaluationContext
  daimo:ParticipantRole ) .
```

Listing 1: Axiomas centrales alrededor de `ExecutionAuthorization`.

Superficie de reutilización. La Tabla 10 resume los vocabularios reutilizados y el uso concreto de cada uno. Tres observaciones son importantes. La primera es que las alineaciones hacia DCAT y ODRL son con los vocabularios estándar, no con entidades internas de ninguna implementación: este punto es relevante porque en la práctica de Eclipse EDC las dos capas se confunden con frecuencia. La segunda es que DAIMO mantiene hacia DSP una relación de mapeo informativo mediante `skos:closeMatch` y `skos:related`, no de subsunción: DSP es un protocolo de mensajería, no una ontología de activos, y subsumir clases DAIMO bajo constructos DSP introduciría tipados espurios. La tercera es que de toda la extensión EDC sólo `edc:ParticipantContext` aparece explícitamente en DAIMO, referenciado por su utilidad para anclar la procedencia agregada de un participante.

Tres subpropiedades que podrían parecer naturales no se declaran, y la ausencia merece documentación explícita. `daimo:authorizesRun` no se declara subpropiedad de `prov:used`; `daimo:grantedTo` no se declara subpropiedad de `prov:qualifiedAssociation`; `daimo:evidenceOf` no se declara subpropiedad de `prov:hadActivity`. En los tres casos, el alineamiento aparentemente intuitivo produciría, bajo razonamiento RDFS u OWL-RL, que las autorizaciones se tipasen como actividades PROV, los agentes receptores como asociaciones reificadas y las evidencias como objetos de influencia. Estos tipados contradicen el modelo conceptual del puente DAIMO. La verificación de implicación (§) se diseñó precisamente para detectar errores de este tipo antes de que contaminen el cierre inferido.

Evaluación

La evaluación distingue dos objetos: DAIMO como artefacto ontológico y la evaluación comparativa de modelos que DAIMO permite describir. El primer objeto se examina como evaluación técnica del artefacto: adecuación a requisitos, consistencia formal, conformidad SHACL, ausencia de pitfalls críticos, comportamiento de los

Tabla 10. Vocabularios reutilizados por DAIMO y uso concreto de cada uno.

Vocabulario	Términos reutilizados	Uso en DAIMO
MLDCAT-AP 3.0.0 (it6:)	MachineLearningModel, Task, Run, Flow, Evaluation, EvaluationMeasure, Benchmark, ComputerInfrastructure, Hardware, Library, HarmRisk, Modality, trainedOn, testedOn	Capa del activo de IA; DAIMO no redefine el modelo.
DCAT 2 (dcat:)	Catalog, Dataset, Distribution, DataService, CatalogRecord, endpointURL	Publicación y descubrimiento.
ODRL 2.2 (odrl:)	Offer, Agreement, Permission, Prohibition, hasPolicy, target, assigner, assignee	Política, oferta y acuerdo.
PROV-O (prov:)	Activity, Entity, Agent, Bundle, Role, wasGeneratedBy, wasAssociatedWith	Procedencia y roles.
SPDX (spdx:)	Checksum, algorithm, checksumValue	Integridad de evidencia de auditoría.
DSP (dspace:)	ContractOffer, ContractNegotiation, TransferProcess	Mapeos informativos skos:closeMatch y skos:related; no subsunción.
EDC (edc:)	ParticipantContext	Única extensión específica de EDC referenciada.

alineamientos y respuesta a preguntas de competencia. El segundo aparece sólo dentro del demostrador como comparación de resultados bajo un contexto compartido, sin constituir un benchmark empírico de modelos reales. En términos de LOT, la validación de propósito se cubre de forma acotada mediante un grafo de instancia realista, consultas SPARQL ejecutables y pruebas negativas; la validación externa con expertos se declara explícitamente como evidencia pendiente para la versión de envío (§). Siguiendo la distinción habitual entre cobertura de requisitos, corrección formal, validación basada en restricciones y utilidad aplicada³³, preguntamos si el perfil representa el intercambio objetivo, si el artefacto OWL es formalmente coherente, si las restricciones SHACL discriminan grafos válidos e inválidos, y si las preguntas de competencia son ejecutables sobre el grafo materializado. Sobre esta distinción, el análisis sigue el patrón de cinco dimensiones de RePlanIT⁹: instanciación del escenario (§), verificación formal mediante razonamiento, comprobación de implicación y escaneo OOPS! (§), validación de instancias frente a SHACL (§), ejecución de las veintitrés preguntas de competencia (§) y verificación de los alineamientos de reutilización (§). Todas las dimensiones son reproducibles mediante cinco scripts Python del paquete público: `validate.py`, `reasoner_check.py`, `oops_check.py`, `tests/negative_test.py` y `scalability_benchmark.py`.

Modelado del caso de uso

El escenario introducido en § se instancia sobre DAIMO como demostrador controlado. El grafo resultante (`examples/flood-risk-scenario.ttl`) contiene 225 tripletas de datos que ejercen al menos una vez cada clase nativa de la ontología y sobre las que se ejecutan las veintitrés consultas de competencia reportadas en §. Todas las organizaciones, nombres de modelo, versiones de

dataset, métricas y endpoints son anónimos o sintéticos. El demostrador no es un estudio de despliegue, un *gold standard* ni una validación con catálogos reales de modelos; su función es probar adecuación representacional y hacer ejecutables los artefactos de validación.

El Proveedor A publica tres ofertas gobernadas en un catálogo federado: dos versiones de un mismo modelo geoespacial de predicción de riesgo y una variante de sólo lectura con política más estricta. Las ofertas comparten tarea y dominio, pero difieren en las condiciones ODRL asociadas. Esta parte del escenario ejercita la distinción entre el modelo como activo de IA descrito con MLDCAT-AP y la oferta como registro contextual de catálogo con condiciones de uso propias.

El modelo seleccionado se despliega sobre un entorno de cómputo descrito y se expone mediante dos interfaces de servicio: un endpoint REST con entrada JSON y autenticación por bearer token, y un endpoint gRPC con protobuf y autenticación mTLS. El uso multi-endpoint ejercita la decisión de modelado según la cual un mismo despliegue puede exponer varios servicios y contratos, en lugar de forzar un despliegue distinto por endpoint.

El Consumidor municipal B obtiene una autorización de ejecución con fechas de vigencia explícitas, invoca el modelo y recibe un artefacto de predicción derivado. La ejecución se vincula con evidencia de auditoría que contiene registro de integridad, firmante y marca temporal, de modo que un actor de gobernanza pueda reconstruir cómo se produjo el artefacto y bajo qué autorización. Un registro de procedencia cruzada agrega la cadena a través de los contextos de proveedor, consumidor y plataforma. El Evaluador D publica dos resultados sintéticos de evaluación bajo el mismo benchmark, protocolo y semilla aleatoria, permitiendo comparar dos versiones del modelo bajo condiciones compartidas. La Figura 3 ilustra el escenario instanciado.

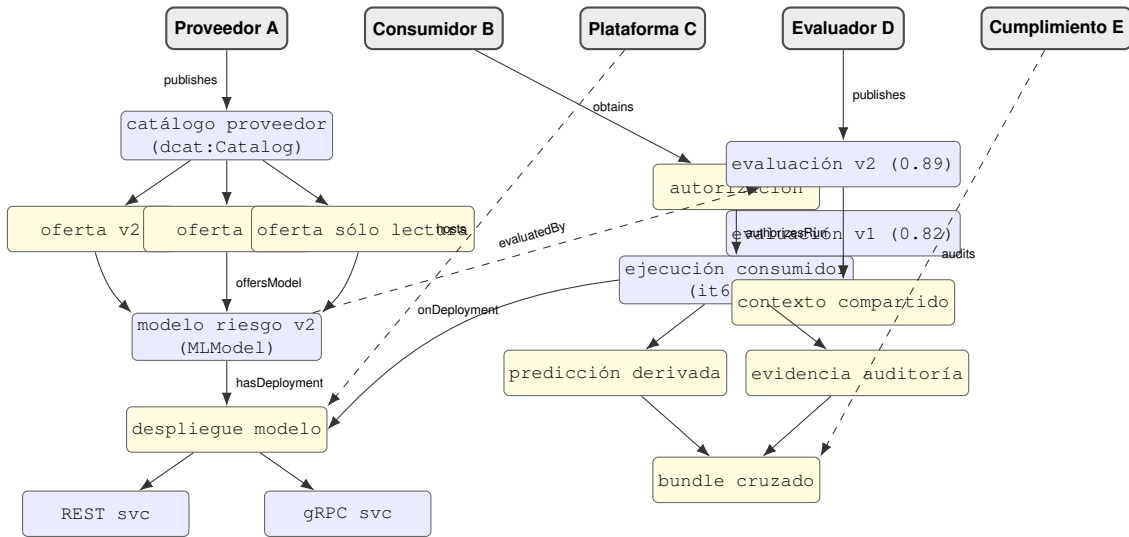


Figura 3. Demostrador anónimo instanciado sobre DAIMO. Los rectángulos grises redondeados de la fila superior son los cinco contextos de participante; las instancias sombreadas en amarillo son constructos de gobernanza introducidos por DAIMO; las instancias en azul claro son recursos descritos con vocabularios reutilizados. Las flechas sólidas denotan relaciones de dominio; las discontinuas muestran el rol funcional del participante sobre la instancia inferior.

Verificación formal

El razonador OWL 2 DL HermiT, invocado mediante `owlready2`, reporta que la unión del núcleo, los alineamientos y el grafo de ejemplo es lógicamente consistente y que no existen clases insatisfactibles; el tiempo de razonamiento reportado es de 1,25 segundos. La materialización OWL-RL (biblioteca `owlrl`) deriva 1170 tripletas adicionales sobre 818 iniciales (1988 totales) en 0,58 segundos, sin producir individuos tipados como `owl:Nothing` ni subclases insatisfactibles. Ambos razonadores coinciden, con HermiT como validador de la corrección lógica y OWL-RL como materializador pragmático para las consultas que requieren inferencia por subpropiedad o subclase.

La consistencia lógica es condición necesaria pero no suficiente para la corrección semántica de una ontología de integración. Una axiomatización puede ser perfectamente consistente y, al mismo tiempo, implicar tipados no intencionados si no declara disyunción explícita con la clase adecuada o si adopta una subpropiedad cuyo rango fuerza una subsunción indeseada. Por eso DAIMO implementa adicionalmente una *verificación de implicación* sobre el cierre OWL-RL: para cada clase nativa, el script `reasoner_check.py` enumera todas las superclases inferidas y las contrasta con una lista de superclases prohibidas. Por ejemplo, `AuditEvidence` no debe ser inferida como `prov:Activity`, porque es una cosa producida por actividades, no una actividad. El reporte cubre las catorce clases nativas (nueve de primer nivel más cinco subclases de `ParticipantRole`) y no emite ninguna advertencia. Esta comprobación es complementaria a la de consistencia y captura errores que un razonador DL acepta porque son lógicamente consistentes aunque semánticamente incorrectos.

El escáner OOPS!³⁴, consultado por API REST sobre la unión del núcleo y los alineamientos, reporta cero pitfalls Críticos, cero Importantes y dos Menores. El Menor P13 (“relaciones inversas no declaradas”) afecta a

34 elementos, y corresponde a propiedades para las que no existe una inversa semánticamente significativa (por ejemplo, `daimo:integrityHash: nombrar una inversa “tiene hash”` no añade capacidad de consulta). El Menor P04 (“elementos no conectados”) afecta a siete elementos que son clases externas declaradas localmente para permitir que el razonador las interprete sin exigir que el archivo las defina completamente. Ambos Menores son características habituales de perfiles de integración y no indican defectos de modelado; su aparición es precisamente el resultado de la disciplina *reuse-first* que el perfil aplica.

Las métricas de calidad estructural se interpretan como señales diagnósticas, no como criterios absolutos de aceptación. Conteos como profundidad de jerarquía, amplitud taxonómica, conectividad, equivalencias explícitas o elementos aislados ayudan a localizar zonas que merecen inspección, pero no dictan por sí mismos una corrección automática. En OWL, un ciclo de `rdfs:subClassOf` puede expresar equivalencia implícita antes que un error; una `owl:equivalentClass` puede ser un alineamiento legítimo; y una disyunción sólo debe añadirse cuando el compromiso ontológico es verdadero en todo modelo posible del dominio. DAIMO usa por ello OOPS!, HermiT, OWL-RL, SHACL, pruebas negativas y preguntas de competencia como capas complementarias: cada una observa un tipo de riesgo distinto y ninguna sustituye el juicio de modelado.

Validación SHACL: estructural, invariantes y banco negativo

DAIMO superpone tres familias de *shapes* SHACL, cada una respondiendo a una pregunta de validación distinta. Las *shapes* estructurales preguntan si toda instancia DAIMO-nativa porta las propiedades obligatorias de su clase. Las *shapes* de conformidad preguntan si las clases reutilizadas externas satisfacen las obligaciones adicionales que el perfil les impone. Las invariantes SPARQL preguntan si las relaciones entre oferta, política, despliegue, autorización, ejecución, artefacto derivado y evidencia siguen siendo

coherentes. Juntas, las tres familias llevan la validación de lo sintáctico a lo semántico: de “¿está esta instancia bien formada?” a “¿es este grafo internamente consistente?”. La capa SHACL funciona así como contrato de datos del perfil DAIMO; no introduce una ontología paralela ni exige trazabilidad, formatos de modelo o métricas universales que no estén justificadas por las preguntas de competencia.

En términos de implementación, las restricciones se expresan como `sh:NodeShape` dirigidas a clases DAIMO o clases reutilizadas bajo el perfil, y como `sh:PropertyShape` internas que fijan cardinalidades, tipos de dato, clases esperadas, patrones o enumeraciones cerradas. En un flujo operativo, un proveedor podría publicar metadatos RDF o JSON-LD de un modelo, un conector o servicio de catálogo ejecutaría la capa `daimo-shapes.ttl`, y el `sh:ValidationReport` resultante indicaría si el grafo conforma o qué propiedad concreta incumple el perfil. Este flujo describe el modo esperado de consumo de las *shapes*; no se presenta como un conector productivo ya desplegado ni como una validación de la ontología TBox en sí misma.

Las nueve *shapes* estructurales imponen completitud mínima por clase; el grafo positivo de 225 tripletas satisface `conforms=True`. Las tres *shapes* de conformidad añaden obligaciones operativas sobre clases reutilizadas: `OfferInDAIMOShape` exige que toda `odrl:Offer` utilizada en DAIMO declare `odrl:assigner` y `odrl:target` (a nivel de política o por permiso, mediante el conectivo `sh:or`); `MachineLearningModelInDAIMOShape` exige política ODRL, título e identificador sobre `it6:MachineLearningModel`; `RunInDAIMOShape` exige *flow*, algoritmo, agente asociado y marca temporal sobre `it6:Run`. Adicionalmente, `sh:in` restringe los valores de `daimo:authMethod` a un vocabulario cerrado de nueve tokens (`none`, `api-key`, `basic`, `bearer`, `oauth2-bearer`, `oauth2-client-credentials`, `mtls`, `jwt`, `dataspace-token`), y `sh:pattern` restringe `daimo:protocol` a un patrón que acepta los nombres habituales de protocolo de evaluación (`holdout`, `stratified-holdout`, `leave-one-out`, `temporal-split`, `<n>-fold-cv`, `stratified-<n>-fold-cv`, `bootstrap-<n>`).

Sobre este estrato estructural se sitúan las seis invariantes SHACL-SPARQL entre clases, codificadas mediante `sh:SPARQLConstraint`, que son el elemento diferenciador de DAIMO frente a un perfil puramente descriptivo. La invariante INV-1 garantiza la consistencia de la cadena de autorización de derivación: todo `DerivedArtifact` debe portar una propiedad `daimo:underAuthorization` cuya autorización cubra precisamente la ejecución de la que el artefacto se deriva. La INV-2 impone que el agente asociado a una ejecución mediante `prov:wasAssociatedWith` aparezca también como `daimo:grantedTo` en alguna de las autorizaciones que cubren esa ejecución, haciendo explícito que el actor registrado en PROV y el beneficiario de la autorización ODRL deben ser la misma entidad. La INV-3 verifica la coherencia del plano de datos: el `dcat:DataService`

expuesto por un `ModelDeployment` debe servir exactamente el modelo que el despliegue declara desplegar, eliminando la posibilidad de que un endpoint anuncie un modelo pero sirva otro. La INV-4 exige que el `daimo:expiresAt` de cada `ExecutionAuthorization` sea estrictamente posterior al `prov:startedAtTime` de toda ejecución que la autorización cubre, reflejando que una autorización no puede justificar ejecuciones que comenzaron tras su caducidad. La INV-5 requiere que el modelo referenciado por `daimo:offersModel` en una `AIAssetOffering` aparezca como `odrl:target` de la política asociada, a nivel de política o en alguno de sus permisos; si la política no gobierna el modelo registrado, la cadena de gobernanza está rota aunque ambos recursos existan. La INV-6 cierra el mismo triángulo por el lado del proveedor: el `daimo:offeredBy` de toda oferta debe coincidir con el `odrl:assigner` de su política, impidiendo inconsistencias entre la atribución del registro de catálogo y la autoría declarada en la oferta ODRL. El grafo positivo conforma las seis invariantes sin advertencias. El Listado 2 muestra la codificación concreta de INV-5, ilustrando el patrón `FILTER NOT EXISTS` que captura la disyunción entre política global y permiso individual.

```
daimo:OfferingPolicyTargetInvariant a sh:NodeShape
;
sh:targetClass daimo:AIAssetOffering ;
sh:sparql [
  a sh:SPARQLConstraint ;
  sh:message "INV-5 violation: offering's
             offersModel does not
             appear as odrl:target of the
             attached policy."@en ;
  sh:prefixes daimo:_invariantPrefixes ;
  sh:select """
    SELECT $this ?model ?policy WHERE {
      $this daimo:offersModel ?model ;
      daimo:hasOfferPolicy ?policy .
      FILTER NOT EXISTS { ?policy odrl:
        target ?model }
      FILTER NOT EXISTS { ?policy odrl:
        permission/odrl:target ?model }
    }
    """ ;
]
```

Listing 2: Invariante INV-5 como `sh:SPARQLConstraint`.

Las *shapes* que conforman sobre el grafo positivo son necesarias pero no suficientes para demostrar que las restricciones son semánticamente discriminantes: una restricción trivialmente satisfacible sobre cualquier grafo pasaría la validación positiva sin aportar garantías reales. Por eso DAIMO publica un grafo complementario `tests/negative-examples.ttl` (118 tripletas) que codifica seis casos deliberadamente inválidos —`bad:INV1-artifact`, `bad:INV2-run`, `bad:INV3-deployment`, `bad:INV4-auth`, `bad:INV5-offering` y `bad:INV6-offering`—, cada uno construido para violar exactamente una invariante. El script `tests/negative_test.py` carga la ontología, los *shapes* y el grafo negativo, ejecuta SHACL y verifica dos condiciones: que la validación falle globalmente y que cada nodo focalizado esperado

aparezca en el informe con el mensaje específico de la invariante violada. Las seis invariantes se disparan correctamente sobre su violación correspondiente. Esta evidencia, complementaria a la validación positiva, es la que permite afirmar con fundamento que las invariantes capturan reglas de negocio reales.

Preguntas de competencia

Las veintitrés preguntas de competencia se expresan como consultas SPARQL (daimo/queries/queries.md) y se ejecutan sobre el cierre OWL-RL materializado del grafo positivo. La Tabla 11 resume la cobertura por categoría, los conteos observados y las consultas que requieren razonamiento explícito; el conjunto completo de consultas aparece en el Apéndice . Las veintitrés devuelven al menos una fila, lo que verifica que el grafo de demostración contiene la información necesaria para responder a cada pregunta con las primitivas que DAIMO declara. Este resultado se interpreta como validación funcional de cobertura, no como prueba única de validez ontológica: se complementa con HermiT, OWL-RL, OOPS!, SHACL positivo y pruebas negativas. Del mismo modo, la longitud de una consulta o su número de uniones se trata como señal diagnóstica de inspección, no como evidencia automática de sobreingeniería.

Dos resultados conviene señalar. Las consultas CQ-R2 y CQ-E1 dependen de la relación `subPropertyOf` entre `daimo:offersModel` y `foaf:primaryTopic` y no devuelven filas sobre un motor SPARQL puro sin razonamiento. La dependencia es documentada, no un defecto: alinear `offersModel` bajo `foaf:primaryTopic` hace las ofertas descubribles mediante la propiedad DCAT estándar, beneficio que sólo se materializa bajo inferencia. Un consumidor que necesite ejecutar esas consultas sobre un endpoint puro puede materializar el cierre previamente o reescribir las consultas enumerando ambas propiedades. El segundo resultado es que CQ-G2 devuelve dos filas — una por endpoint del despliegue multi-protocolo REST y gRPC —, lo que confirma el modelado de múltiples servicios por despliegue como patrón legítimo y directamente consultable desde SPARQL.

Dos fragmentos ilustran el patrón de formalización. El Listado 3 muestra CQ-G3: la consulta recupera la autorización que cubre una ejecución concreta, el beneficiario autorizado y la fecha de expiración. La alineación `ExecutionAuthorization` $\sqsubseteq$ `odrl:Agreement` hace que ese recurso sea interpretable como acuerdo ODRL sin duplicar propiedades de contrato en la consulta. El Listado 4 muestra CQ-V3: dos evaluaciones se comparan sólo porque comparten el mismo `SharedEvaluationContext`; el ordenamiento por valor de métrica es significativo dentro de ese contexto y no como ranking global independiente del dataset, protocolo o semilla.

```
SELECT ?auth ?grantee ?expires WHERE {
  ?auth a daimo:ExecutionAuthorization ;
    daimo:authorizesRun
      <https://example.org/daimo-scenario/run-2026-04-20-legs> ;
    daimo:grantedTo ?grantee ;
    daimo:expiresAt ?expires .
}
```

```
}
```

Listing 3: CQ-G3: autorización de una ejecución concreta.

```
SELECT ?model ?value WHERE {
  ?eval a it6:Evaluation ;
    it6:evaluates ?model ;
    daimo:usesEvaluationContext
      <https://example.org/daimo-scenario/evalctx-climatebench-v1-2026-1-holdout> ;
    it6:hasEvaluationMeasure ?m .
  ?m it6:hasValue ?value .
}
ORDER BY DESC(?value)
```

Listing 4: CQ-V3: ranking de modelos bajo un contexto compartido.

Reutilización verificada. La reutilización se verifica directamente sobre el módulo de alineamientos. Ocho `rdfs:subClassOf` y cinco `rdfs:subPropertyOf`, listados en la Tabla 9 y visualizados en la Figura 2, conectan DAIMO con sus cinco vocabularios parentales (DCAT, MLDCAT-AP, ODRL, PROV-O y FOAF). La sexta subpropiedad DAIMO hacia un vocabulario externo, `hasOffering` $\sqsubseteq$ `foaf:isPrimaryTopicOf`, se declara en el núcleo porque implementa la navegación inversa de `offersModel`. La verificación opera en dos dimensiones.

La primera es *estructural*: toda clase nativa que podría haberse definido aisladamente se sitúa bajo un padre externo siempre que dicho padre exista, y toda propiedad nueva extiende una propiedad reutilizada siempre que ésta ya cargue el significado previsto. No se introduce ninguna clase por razones estilísticas, y las clases `IOContract` y `SharedEvaluationContext` quedan sin padre externo sólo porque ningún padre con semántica veraz existe (§).

La segunda es *semántica*: los alineamientos no deben producir inferencias no intencionadas. La verificación de implicación de § inspecciona el cierre OWL-RL de cada alineamiento y confirma que ninguno produce una superclase semánticamente incorrecta sobre ninguna clase nativa. El razonador reporta cero advertencias sobre las catorce clases. Tres alineamientos de subpropiedad hacia PROV-O se excluyeron deliberadamente del módulo por el mismo motivo: cada uno habría producido un tipado prohibido bajo cierre RDFS (§).

Cobertura de requisitos no funcionales

Los requisitos no funcionales se evalúan sobre el artefacto ontológico y sus flujos de validación, no sobre una plataforma de ejecución federada. En particular, la escalabilidad se interpreta como crecimiento del número de instancias de intercambio gobernado en el ABox; el núcleo DAIMO permanece estable en catorce clases nativas. Por tanto, el resultado no mide concurrencia, latencia de un conector ni rendimiento productivo de inferencia en línea, sino tratabilidad reproducible de grafos DAIMO crecientes bajo las operaciones que el perfil exige: parseo RDF, materialización OWL-RL, validación SHACL y consultas SPARQL.

Tabla 11. Cobertura de las 23 preguntas de competencia. Cada CQ devuelve el número de filas indicado sobre el grafo positivo cerrado con OWL-RL. Las celdas en negrita señalan las consultas que requieren razonamiento explícito más allá de SPARQL básico.

Categoría	Consultas (filas)	N.	Inferencia requerida
Registro y publicación (R)	R1(3) R2(1) R3(1) R4(2) R5(2)	5	R2 (cadena subPropertyOf), R5 (ruta subClassOf)
Descubrimiento y selección (D)	D1(3) D2(2) D3(4) D4(1)	4	D1 (subtipo de tarea), D2 (coincidencia de política)
Ejecución y auditableidad (E)	E1(4) E2(1) E3(2) E4(1) E5(1)	5	E1 (offersModel $\sqsubseteq$ foaf:primaryTopic), E2 (unión agreement-run)
Evaluación y reproducibilidad (V)	V1(1) V2(1) V3(2) V4(1) V5(4)	5	V2, V3 (agregación)
Puente de gobernanza (G)	G1(1) G2(2) G3(1) G4(1)	4	G4 (agregación GROUP_CONCAT); G3 ejercita directamente ExecutionAuthorization como puente DAIMO-ODRL

Tabla 12. Trazabilidad resumida CQ → clases → propiedades → resultado. Las consultas completas se publican en `daimo/queries/queries.md`; esta tabla permite inspeccionar dentro del PDF qué fragmento semántico ejerce cada CQ.

CQ	Pregunta resumida	Clases principales	Propiedades recorridas	Filas
R1	Modelos publicados por un proveedor	AIAssetOffering, MachineLearningModel	offeredBy, offersModel	3
R2	Oferta, proveedor y fecha de un modelo	AIAssetOffering, CatalogRecord	offersModel $\sqsubseteq$ foaf:primaryTopic, offeredBy, dct:issued	1
R3	Licencia y política aplicables	MachineLearningModel, odrl:Offer	hasOfferPolicy, dct:license, odrl:permission	1
R4	Contrato de entrada/salida	ModelDeployment, IOContract	hasIOContract, inputFormat, outputFormat, authMethod	2
R5	Rol concreto del proveedor	ParticipantRole, subclases de rol	offeredBy, rdf:type/rdfs:subClassOf*	2
D1	Modelos por tarea y dominio	MachineLearningModel, Task	it6:hasTask, dcat:theme	3
D2	Modelos por patrón de política	AIAssetOffering, odrl:Offer	hasOfferPolicy, odrl:permission, odrl:prohibition	2
D3	Endpoints y autenticación	ModelDeployment, DataService, IOContract	exposedAs, endpointURL, authMethod	4
D4	Umbral mínimo de métrica	Evaluation, SharedEvaluationContext	usesEvaluationContext, it6:hasEvaluationMeasure, it6:hasValue	1
E1	Endpoint desde una oferta	AIAssetOffering, ModelDeployment, IOContract	offersModel, hasDeployment, exposedAs, hasIOContract	4
E2	Ejecuciones autorizadas	ExecutionAuthorization, Run	authorizesRun, grantedTo, prov:wasAssociatedWith	1
E3	Implementación e infraestructura	Run, Flow, ComputerInfrastructure	it6:hasFlow, it6:hasAlgorithm, onInfrastructure	2
E4	Evidencia de auditoría	AuditEvidence, spdx:Checksum	evidenceOf, integrityHash, signedBy, recordedAt	1
E5	Artefactos derivados	DerivedArtifact, ExecutionAuthorization	derivedFromRun, underAuthorization	1
V1	Contexto compartido	SharedEvaluationContext, Evaluation	contextTask, contextDataset, protocol, randomSeed	1
V2	Mejor modelo bajo contexto	Evaluation, MachineLearningModel	usesEvaluationContext, it6:hasEvaluationMeasure, it6:hasValue, ORDER BY	1
V3	Ranking bajo misma métrica	Evaluation, SharedEvaluationContext, MachineLearningModel	usesEvaluationContext, it6:evaluates, it6:hasEvaluationMeasure, it6:hasValue, ORDER BY	2
V4	Benchmarks del modelo	MachineLearningModel, Benchmark	it6:hasBenchmark, dct:title opcional	1
V5	Artefactos de reproducibilidad	Flow, AuditEvidence, DerivedArtifact	records, evidenceOf, integrityHash	4
G1	Ofertas que incluyen un modelo	AIAssetOffering, MachineLearningModel	offersModel, hasOfferPolicy	1
G2	Despliegues por modelo	ModelDeployment, IOContract	deploysModel, onInfrastructure, hasIOContract	2
G3	Acuerdo que autorizó una ejecución	ExecutionAuthorization, Run, odrl:Agreement	authorizesRun, grantedTo, expiresAt	1
G4	Bundle completo de procedencia	CrossParticipant ProvenanceRecord, DerivedArtifact	records, spansParticipantContext, GROUP_CONCAT	1

Discusión

DAIMO articula publicación, política, ejecución, procedencia y evaluación en un único perfil de integración que sostiene el intercambio gobernado de modelos a través de fronteras organizativas. Su contribución es semántica y de

gobernanza: no estandariza pesos, arquitecturas internas ni mecanismos de inferencia, sino que hace consultable y validable la relación entre metadatos de modelo, políticas, despliegues, ejecuciones, evidencias y contextos de evaluación. Esta posición la diferencia tanto de ontologías de

Tabla 13. Cobertura de requisitos no funcionales de DAIMO. La columna de estado distingue cumplimiento demostrado en el artefacto de pendientes operativos para el paquete público final.

Requisito	Evidencia reproducible	Estado y riesgo residual
Consistencia lógica	HermiT reporta consistencia y cero clases insatisfactibles; OWL-RL materializa 1 170 tripletas sin owl:Nothing; la verificación de implicaciones inspecciona catorce clases nativas con cero advertencias.	Cumplimiento fuerte. La consistencia se complementa con pruebas negativas SHACL para evitar restricciones triviales.
Interoperabilidad	El perfil usa RDF/OWL 2 DL, SHACL y SPARQL; reutiliza DCAT, MLDCAT-AP, ODRL, PROV-O, FOAF, SKOS y SPDX mediante IRI's externos y alineamientos axiomáticos.	Cumplimiento fuerte en el artefacto. Persisten pendientes operativos de publicación FAIR: w3id, DOI Zenodo y WIDOCO final.
Escalabilidad	scalability_benchmark.py genera grafos sintéticos conformes de 100 y 1 000 unidades de intercambio; ambos conforman SHACL y responden las consultas de control.	Cumplimiento medido y acotado. SHACL es el coste dominante; no se afirma rendimiento productivo ni resultado de 10 000 unidades sin ejecutarlo.
Extensibilidad	El núcleo mantiene alcance mínimo y versionado SemVer; las extensiones se orientan a perfiles complementarios, por ejemplo daimo-discovery, sin absorber lógica de ranking o ejecución.	Cumplimiento fuerte para evolución controlada. Nuevos conceptos deben justificar que no existen en vocabularios reutilizados.
Modularidad	Núcleo, alineamientos, shapes, ejemplos, consultas, pruebas negativas y reportes están separados; cada módulo puede ejecutarse o revisarse de forma independiente.	Cumplimiento fuerte. La independencia modular reduce el riesgo de romper el núcleo al añadir perfiles externos.
Documentación	Cada término incluye etiquetas y definiciones SKOS; ONTOLOGY-REFERENCE.md, VALIDATION-MATRIX.md y REPRODUCIBILITY.md trazan clases, propiedades, CQs, invariantes y comandos.	Cumplimiento fuerte localmente. Antes del envío debe regenerarse la documentación pública WIDOCO y cerrar metadatos ORCID/DOI.

Tabla 14. Benchmark sintético de escalabilidad acotada. Cada unidad contiene una oferta, modelo, política ODRL, despliegue, servicio, contrato de I/O, autorización, ejecución, artefacto derivado, evidencia de auditoría, evaluación y bundle de procedencia. Reporte completo: reports/scalability-benchmark.md.

Unidades	Tripletas datos	Tripletas cierre	Parseo (s)	OWL-RL (s)	SHACL (s)	SPARQL (s)	Conforma
100	8 053	16 248	0,468	5,478	10,374	0,080	Sí
1 000	80 053	147 648	4,660	53,672	135,010	0,356	Sí

dominio publicadas recientemente —RePlanIT⁹, GloSIS¹⁰ o la ontología de mantenimiento¹¹— como de las ontologías previas del ciclo de vida de aprendizaje automático. Con las primeras comparte la ética de ingeniería ontológica —reutilización disciplinada, implementación ejecutable y evaluación contra preguntas de competencia— pero la aplica a la articulación entre catálogo de IA y plano de datos. Con las segundas comparte el dominio de aplicación pero suma la capa de gobernanza que ellas no abordan. Tres rasgos de diseño diferencian la ontología. El primero es la verificación de implicación sobre el cierre OWL-RL, que complementa al razonamiento de consistencia enumerando las superclases inferidas de cada clase nativa y detectando alineamientos semánticamente incorrectos que un razonador DL acepta sin contradicción. El segundo son las invariantes SHACL-SPARQL entre clases, acompañadas de un banco de pruebas negativas que confirma su carácter discriminante: cada invariante se dispara sobre un caso diseñado para violarla, no sólo sobre el grafo positivo. El tercero es la orientación *reuse-first*, encarnada en alineamientos axiomáticos explícitos y en la decisión documentada de no declarar tres subpropiedades hacia PROV-O cuya adopción habría producido tipados no intencionados sobre clases de artefacto.

Las limitaciones del perfil se agrupan en tres líneas argumentales. La primera concierne al *nivel de detalle semántico* de varias extensiones DAIMO. Un límite transversal es el dinamismo del dominio: nuevas arquitecturas, formatos de despliegue, métricas y prácticas de evaluación evolucionan más rápido que los ciclos de mantenimiento de una ontología, por lo que deben entrar mediante versionado y perfiles complementarios antes que como ampliaciones precipitadas del

núcleo. DAIMO cubre el descubrimiento de modelos de IA como activos gobernados: las CQs D1–D4 localizan modelos por tarea, dominio, política, endpoint y umbral de evaluación. No pretende ser un catálogo de catálogos ni un motor de consulta federada, ni cubrir por sí mismo el descubrimiento de modelos de datos, esquemas o vocabularios sectoriales. El IOContract captura el formato de entrada/salida y el método de autenticación; este último está restringido por sh:in a un vocabulario cerrado. Las referencias opcionales inputSchema, outputSchema y perfiles basados en dct:conformsTo pueden apuntar a esquemas o estándares externos, pero la compatibilidad semántica fina entre los esquemas del modelo y del consumidor queda fuera del núcleo y debe abordarse mediante perfiles complementarios, como daimo-discovery, o shapes específicas del espacio de datos. Del mismo modo, DAIMO describe ejecuciones autorizadas y auditables, no implementa enforcement federado: el uso de conectores, nodos neutrales, contenedores o TEE pertenece a la plataforma y puede reflejarse mediante metadatos o perfiles, no como axiomas universales. El SharedEvaluationContext reifica la combinación tarea-dataset-versión-protocolo-semilla suficiente para comparaciones homogéneas sobre una métrica única, pero no cubre protocolos heterogéneos de *benchmarking*, conjuntos compuestos de métricas ni equivalencia matemática de indicadores calculados por procedimientos distintos. Las métricas de sesgo, equidad, seguridad o interpretabilidad pueden representarse como medidas de evaluación cuando existen y están contextualizadas; DAIMO no define una teoría universal de esos conceptos. Los atributos protegidos, técnicas de mitigación y auditorías de sesgo pertenecen a

perfiles de cumplimiento o de dominio. El modelado de roles conflata el tipo (`ModelProvider`) con la asignación — la tupla agente-rol-contexto-alcance—, confluencia que OntoClean recomienda separar. MLDCAT-AP adopta la misma pragmática en `hasRegisteredUser`, lo que sitúa la contextualización fina en el horizonte del refinamiento futuro, no en el del error de diseño.

La segunda línea concierne a los *compromisos de integración* y a los efectos que tienen sobre consumidores que combinen DAIMO con vocabularios adyacentes. La propiedad `daimo:records` declara `rdfs:range prov:Activity`: bajo razonamiento RDFS, las evaluaciones MLDCAT-AP agrupadas dentro de un `CrossParticipantProvenanceRecord` son inferidas como actividades PROV aunque MLDCAT-AP no declare esa subsunción. El efecto es inocuo para consultas de auditoría, pero un consumidor que integre DAIMO, MLDCAT-AP y PROV-O puede obtener tipados sorprendentes sobre `it6:Evaluation`. En la misma línea, DAIMO no se ancla a una ontología de nivel superior (BFO, GFO, DOLCE o UFO): la ausencia responde a que un perfil de integración no es una ontología de referencia. Un trabajo futuro coherente sería acomodar la taxonomía a una de esas ontologías superiores sin renunciar a la reutilización directa de los vocabularios de dominio que motivan DAIMO.

Una variante práctica de este problema aparece en la soberanía de datos: puede ser necesario propagar restricciones desde un dataset de entrenamiento hacia el modelo o hacia sus artefactos derivados. DAIMO no axiomatiza esa propagación como regla universal, porque un modelo no hereda automáticamente todas las restricciones jurídicas o territoriales de sus datos de origen; la respuesta depende de licencia, contrato, jurisdicción, riesgo de memorización, técnica de entrenamiento y política acordada. Lo que DAIMO sí aporta es la estructura para representarla y auditarla cuando una organización la exige: `it6:trainedOn` y `daimo:contextDataset` conectan modelo, evaluación y dataset; ODRL expresa restricciones de uso; `ExecutionAuthorization` vincula esas condiciones con ejecuciones concretas; y la evidencia de auditoría junto con el registro de procedencia cruzada permiten reconstruir si el intercambio respetó la política. Reglas de propagación más estrictas deben implementarse como perfiles SHACL u ODRL específicos del espacio de datos, no como axiomas generales del núcleo.

La tercera línea concierne a las *condiciones de reproducibilidad y validación externa*. Los resultados numéricos reportados dependen de un perfil de razonamiento concreto: el modo `inference=rdfs` en `pyshacl`, necesario para que las subpropiedades implicadas se materialicen antes de evaluar los *shapes* de conformidad, y el cierre OWL-RL materializado antes de las consultas de competencia. Esta dependencia está documentada pero es una condición que un consumidor debe conocer para reproducir los resultados exactos. La versión actual reporta validación ejecutable; la versión de envío debe incorporar una validación externa ligera con tres perfiles, diseñada como revisión cualitativa estructurada y no como experimento estadístico. Para evitar que el juicio experto sea una revisión informal, los materiales deben separarse por audiencia: glosario normalizado,

preguntas de competencia, diagrama conceptual y escenario anónimo para expertos de dominio; extractos OWL, alineamientos y *shapes* para el perfil Semantic Web. Las preguntas dirigidas a expertos no sustituyen a las preguntas de competencia de LOT: las CQs se verifican mediante SPARQL sobre el grafo de demostración, mientras que la revisión experta juzga suficiencia, claridad, alineación con prácticas de espacio de datos y utilidad para MLOps o cumplimiento. La aceptación no exige unanimidad ni una cobertura nominal del 100%; exige que toda observación quede trazada como cambio aceptado, cambio rechazado con justificación, fuera de alcance o trabajo futuro, junto con su impacto sobre clases, propiedades, CQs o restricciones. La Tabla 15 fija el protocolo mínimo y añade cumplimiento como perfil opcional recomendado cuando el artículo mantenga referencias explícitas al AI Act, sesgo o responsabilidad.

Conviene delimitar la relación entre DAIMO y MLDCAT-AP para disipar toda preocupación de duplicidad. El modelo propiamente dicho se reutiliza directamente mediante `it6:MachineLearningModel`, sin redefinición. La aportación de DAIMO se concentra en la articulación entre catálogo y plano de datos: las cuatro preguntas de competencia de la categoría de gobernanza ejercitan exclusivamente clases DAIMO-nativas y no pueden responderse sólo con MLDCAT-AP, pues requieren `AIAssetOffering` (CQ-G1), `ModelDeployment` (CQ-G2), `ExecutionAuthorization` (CQ-G3) y `CrossParticipantProvenanceRecord` (CQ-G4). Los dos perfiles son complementarios.

Esta delimitación también sitúa la relación con aplicaciones de descubrimiento como `ai-model-discovery`. DAIMO no debe absorber la lógica de ranking, recomendación, ejecución federada o interfaz de usuario de una aplicación concreta; esas capacidades pertenecen a una capa operacional que consume el grafo DAIMO, no al núcleo ontológico. Una evolución razonable sería definir un perfil complementario `daimo-discovery` que añada metadatos de indexación, señales agregadas de popularidad, latencia observada, huella computacional o sostenibilidad, siempre como extensiones derivables o asociadas a contextos de evaluación, sin modificar las catorce clases nativas del núcleo. De este modo, el proyecto `ai-model-discovery` puede actuar como aplicación futura y banco de explotación del perfil, mientras que DAIMO conserva un alcance publicable como ontología de integración para intercambio gobernado.

La misma regla aplica a las preguntas complementarias sobre disponibilidad, reproducibilidad profunda y confianza operacional. Varias de ellas ya están cubiertas por el perfil actual porque forman parte de la cadena catálogo-política-despliegue-ejecución; otras pertenecen al plano del activo MLDCAT-AP o a perfiles futuros. La Tabla 16 resume la decisión de alcance. Esta tabla no introduce módulos ontológicos nuevos: documenta qué preguntas son núcleo DAIMO y cuáles deben evolucionar como perfiles especializados para no complejizar la contribución principal antes de la sumisión.

Más allá del artefacto presente, DAIMO ilustra un patrón que probablemente se repetirá allí donde participantes autónomos necesiten intercambiar recursos de alto valor sujetos a política. El patrón tiene tres elementos recurrentes: un registro de oferta reificado sobre el propio recurso, una

Tabla 15. Protocolo mínimo de validación externa para cerrar el envío SWJ. La tabla no reporta resultados ya obtenidos; fija la evidencia que debe incorporarse antes de la sumisión.

Perfil	Objeto de revisión	Pregunta principal	Salida esperada
Ontologías / Semantic Web	Modularidad, alineamientos, CQs, SHACL	¿Los compromisos ontológicos son suficientes y no redundantes frente a vocabularios reutilizados?	Lista de ajustes de modelado y documentación.
Data spaces / EDC	Oferta, autorización, contexto de participante, DSP/ODRL	¿La cadena publicación-negociación-invocación-auditoría representa un intercambio gobernado realista?	Observaciones sobre integración con conectores y contratos.
ML / MLOps	Modelo, ejecución, evaluación, reproducibilidad	¿Las entidades de ejecución y comparación son útiles para gestión práctica de modelos?	Observaciones sobre evaluación, latencia, métricas y límites.
Gobernanza / cumplimiento (opcional)	Políticas, evidencia, roles y trazabilidad regulatoria	¿La evidencia DAIMO es suficiente para sostener auditorías de uso, responsabilidad y riesgo sin prometer cumplimiento legal automático?	Observaciones sobre alcance regulatorio, sesgo, explicabilidad y límites jurídicos.

Tabla 16. Cobertura de familias complementarias de preguntas y ruta de evolución. El criterio es mantener el núcleo DAIMO limitado al puente catálogo-dataspace y desplazar detalles de MLOps, seguridad o cumplimiento fino a perfiles especializados.

Familia	Cobertura en DAIMO actual	Ruta de extensión controlada
Disponibilidad del recurso	Endpoint mediante <code>ModelDeployment</code> , <code>dcat:DataService</code> y <code>dcat:endpointURL</code> ; política y licencia mediante <code>ODRL</code> y <code>dct:license</code> ; distribuciones descargables mediante <code>DCAT</code> cuando el modelo se publica como archivo; proveedor mediante roles DAIMO y contexto de participante. <code>dct:publisher</code> queda reservado para el mantenedor del registro o catálogo cuando proceda.	Mantener en el núcleo sólo la disponibilidad necesaria para descubrimiento e invocación. Formatos de artefacto como <code>ONNX</code> o <code>PyTorch</code> se expresan con <code>dct:format/dcat:Distribution</code> o con <code>IOContract</code> si afectan a la invocación.
Reproducibilidad técnica	Dataset de entrenamiento y evaluación mediante <code>MLDCAT-AP</code> , <code>DCAT</code> y <code>PROV-O</code> ; artefactos de reproducibilidad mediante <code>it6:Flow</code> , evidencia, checksum y contexto compartido de evaluación.	Hiperparámetros, dependencias software, versiones de framework, GPU y memoria pertenecen a <code>daimo-reproducibility</code> o a reutilización más detallada de <code>ML-Schema/MEX/MLDCAT-AP</code> , no a nuevas clases núcleo.
Versionado y linaje de modelo	DAIMO puede representar derivación y revisión mediante <code>PROV-O</code> cuando el grafo lo declara, y puede auditar la cadena ejecución-artefacto-autorización.	Un perfil de linaje puede añadir reglas <code>SHACL</code> para <code>prov:wasDerivedFrom</code> , <code>prov:wasRevisionOf</code> , estado de obsolescencia y trazas de reentrenamiento sin alterar las catorce clases nativas.
Automatización MLOps	Fuera del núcleo. DAIMO define el contrato semántico que una herramienta puede poblar, pero no implementa conectores ni extracción automática.	Integradores desde Hugging Face, <code>MLflow</code> o registros de modelos pueden poblar grafos DAIMO y servir como validación aplicada posterior.
Monitorización y deriva	Fuera del núcleo salvo que los resultados se publiquen como evaluaciones contextualizadas.	Un perfil <code>daimo-monitoring</code> puede modelar deriva, degradación, latencia observada, huella computacional o sostenibilidad como métricas temporales o eventos de evaluación.
Seguridad y cumplimiento	El núcleo cubre política, autorización, evidencia de auditoría, checksum, firmante y timestamp. No modela vulnerabilidades, ataques o interpretación jurídica automática.	Perfiles <code>daimo-security</code> y <code>daimo-compliance</code> pueden añadir vulnerabilidades, firmas, controles AI Act, sesgo/equidad/explicabilidad y reglas <code>SHACL/ODRL</code> específicas de dominio.

autorización reificada sobre el permiso, y un bundle de procedencia que cruza contextos administrativos. Conjuntos de datos genómicos, herramientas de apoyo a la decisión clínica, gemelos digitales y modelos industriales presentan cadenas análogas de publicación-invocación-auditoría. DAIMO es específica de IA en lo que reutiliza (`MLDCAT-AP`), pero agnóstica al dominio en los constructos que añade;

confiamos en que esos constructos sirvan como punto de partida para perfiles hermanos en dominios adyacentes.

La ruta de adopción es concreta aunque el artículo no reporte todavía un piloto desplegado. En el plano del activo de IA, DAIMO puede coevolucionar con el dossier `SEMIC/MLDCAT-AP` preservando su capa de descripción de modelos y añadiendo únicamente el puente de gobernanza

hacia espacios de datos. En el plano de intercambio, Eclipse Dataspace Components es el candidato natural de integración: los contextos de participante anclan procedencia agregada, y los acuerdos ODRL ya aparecen al final de una negociación DSP. En el plano de cumplimiento, la evidencia de auditoría DAIMO contiene los elementos que inspecciones alineadas con Gaia-X o con el Anexo IV del AI Act esperarían encontrar: checksum, firmante, timestamp, autorización y vínculo con un bundle de procedencia cruzada. Estas observaciones definen tres rutas de validación posteriores: instanciación sobre un catálogo MLDCAT-AP público, co-despliegue con un conector EDC en un piloto federado y mapeo de un caso de alto riesgo del AI Act contra las seis invariantes SHACL-SPARQL.

Disponibilidad y sostenibilidad

DAIMO se distribuye como paquete público bajo licencia CC-BY 4.0 para la ontología y su documentación, y Apache 2.0 para los scripts de validación. La Tabla 17 resume los elementos entregables y sus ubicaciones. El procedimiento operativo de liberación se documenta en `DEPLOYMENT.md`, de modo que cualquier mantenedor futuro puede ejecutarlo sin conocimiento implícito adicional. Los elementos marcados como “pendiente de cierre” deben resolverse antes de la sumisión final a SWJ, pero no impiden reproducir localmente las validaciones.

El flujo de reproducibilidad no requiere el despliegue previo de la URL persistente: cualquier usuario con Python 3.12 y las dependencias declaradas (`rdflib`, `pyshacl`, `owlrl`, `owlready2`) puede ejecutar los cinco scripts localmente y reproducir los reportes de `reports/`. La URL persistente y el DOI son requisitos FAIR adicionales que complementan pero no condicionan la reproducibilidad. La cabecera de la ontología declara `dct:conformsTo` hacia OWL 2 y utiliza `rdfs:seeAlso` para enlazar los tres módulos y la documentación HTML, de modo que un consumidor puede recorrer la cadena identificador-documento sin cargar el paquete completo.

La sostenibilidad del artefacto se estructura según la Fase 4 de LOT. El fichero `CONTRIBUTING.md` documenta el flujo de triaje de *issues*, el ciclo de pull requests y la política de versionado según SemVer: las versiones *patch* se reservan para correcciones de definición textual o de documentación, las *minor* para nuevas clases o propiedades compatibles y las *major* para cambios que alteren alineamientos o invariantes. Cada cambio no trivial debe declarar qué actividades LOT se reabren de forma acotada y qué impacto tiene sobre preguntas de competencia, *shapes* SHACL, axiomas de alineamiento, grafos ya instanciados e interoperabilidad entre nodos del espacio de datos. Una integración continua pública es un cierre de ingeniería deseable para ejecutar automáticamente sintaxis RDF, Hermit/OWL-RL, SHACL, OOPS! y pruebas negativas en cada cambio; no se presenta como evidencia adicional mientras no esté desplegada. La política de deprecación impone al menos una versión *minor* de coexistencia antes de retirar un término, con `owl:deprecated` y `skos:closeMatch` hacia el término de reemplazo.

El paquete satisface los cuatro principios FAIR. Es *findable* a través del namespace persistente `w3id.org`,

con IRIs por versión y metadatos Dublin Core completos. Es *accessible* mediante negociación de contenido y cinco serializaciones pre-generadas bajo licencia CC-BY 4.0 abierta. Es *interoperable* por su compromiso con vocabularios W3C estándar (DCAT, ODRL, PROV-O, SKOS, FOAF, DCMI) mediante alineamientos axiomáticos explícitos en perfil OWL 2 DL. Es *reusable* gracias a definiciones y ejemplos SKOS por término, metadatos de procedencia declarados, versionado SemVer y las *shapes* SHACL como contrato de datos.

Conclusiones

DAIMO conecta MLDCAT-AP, DCAT, ODRL, PROV-O y el protocolo DSP para convertir los modelos de IA en activos gobernables de primera clase dentro de un espacio de datos. Su contribución específica es la capa de gobernanza ausente entre el catálogo de IA y el plano de datos —registro con política ODRL coherente, autorización de ejecución anclada a un acuerdo, despliegue como puente tipado entre el modelo y su servicio, y procedencia agregada entre contextos administrativos—, capa que los vocabularios reutilizados aisladamente no proporcionan y que cuatro de las veintitrés preguntas de competencia (la categoría G) permiten verificar de forma concreta. El caso de estudio multi-participante ilustra que la ontología puede instanciarse coherentemente sobre un escenario verosímil y responder las cinco familias de consultas operativas que un espacio de datos gobernado plantea. Las líneas de trabajo futuro se derivan directamente de las limitaciones acotadas: instanciación contra catálogos MLDCAT-AP reales, evaluación cualitativa con expertos del dominio, refinamiento del IOContract hacia compatibilidad semántica de esquemas y estándares sectoriales, separación rol-tipo/rol-asignación siguiendo OntoClean, extensión del `SharedEvaluationContext` a protocolos heterogéneos de *benchmarking*, población automática desde registros de modelos y perfiles complementarios de reproducibilidad, monitorización, seguridad y cumplimiento. El artefacto se distribuye como paquete público con namespace persistente, documentación WIDOCO, capa SHACL independiente y suite de validación reproducible, preparado para ser adoptado, extendido o criticado por la comunidad con la misma disciplina metodológica con la que ha sido construido.

Referencias

- Franklin, M., Halevy, A. and Maier, D. (2005) From Databases to Dataspace: A New Abstraction for Information Management. *SIGMOD Record*, 34(4):27–33.
- Albertoni, A. et al. (2020) *Data Catalog Vocabulary (DCAT) — Version 2*. W3C Recommendation.
- Lebo, P., Sahoo, S. and McGuinness, D. (eds.) (2013) *PROV-O: The PROV Ontology*. W3C Recommendation.
- Iannella, R., Guth, S. and Steyskal, R. (eds.) (2018) *ODRL Information Model 2.2*. W3C Recommendation.
- Knublauch, H. and Kontokostas, D. (eds.) (2017) *Shapes Constraint Language (SHACL)*. W3C Recommendation.
- Mitchell, M. et al. (2019) Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*.

Tabla 17. Disponibilidad del artefacto DAIMO como paquete público reproducible.

Elemento	Ubicación
Namespace persistente	https://w3id.org/pionera/daimo ; configuración de negociación de contenido preparada en <code>w3id-redirect/.htaccess</code> . Estado: pendiente de merge en <code>perma-id/w3id.org</code> .
Módulos versionados	Tres ontologías independientes (núcleo, <i>shapes</i> , alineamientos) bajo https://w3id.org/pionera/daimo/ con <code>owl:versionIRI</code> y <code>owl:priorVersion</code> explícitos
Repositorio público	Paquete de código y documentación preparado; la URL definitiva del repositorio público debe fijarse antes del envío final siguiendo <code>DEPLOYMENT.md</code> .
Identificación de autoría	ORCID(s) de los autores pendientes de confirmar e incorporar en la versión de sumisión y en <code>CITATION.cff</code> .
Archivado versionado (DOI)	Release versionada preparada para archivado en Zenodo; el DOI final debe incorporarse en la versión de sumisión.
Serializaciones	Turtle, RDF/XML, JSON-LD, N-Triples (pre-generadas por WIDOCO)
Documentación HTML	WIDOCO 1.4.25 con WebVOWL integrado; negociación de contenido vía <code>.htaccess</code> bajo <code>w3id-redirect/</code>
Referencia humana-legible	ONTOLOGY-REFERENCE.md (clase por clase con <code>skos:definition</code> , <code>skos:example</code> , criterios de identidad y etiquetas OntoClean) y VALIDATION-MATRIX.md
Metadatos de <i>release</i>	<code>CITATION.cff</code> , <code>.zenodo.json</code> y <code>CHANGELOG.md</code>
Paquete de validación	<code>validate.py</code> , <code>reasoner_check.py</code> , <code>oops_check.py</code> , <code>tests/negative_test.py</code>
Gobernanza de mantenimiento	<code>CONTRIBUTING.md</code> (Fase 4 LOT: triaje de <i>issues</i> , pull requests, SemVer, política de deprecación)

7. Arnold, M. et al. (2019) FactSheets: Increasing Trust in AI Services through Supplier's Declarations of Conformity. *IBM Research Report*.
8. Werbrouck, J. et al. (2024) ConSolid: A federated ecosystem for heterogeneous multi-stakeholder projects. *Semantic Web*, 15:429–460.
9. Kurteva, A. et al. (in press) RePlanIT Ontology for Digital Product Passports of ICT: Laptops and Data Servers. *Semantic Web*.
10. Palma, R. et al. (in press) GloSIS: The Global Soil Information System Web Ontology. *Semantic Web*.
11. Woods, C. et al. (in press) An ontology for maintenance activities and its application to data quality. *Semantic Web*.
12. Penteado, B.E., Maldonado, J.C. and Isotani, S. (2023) Methodologies for publishing linked open government data on the Web. *Semantic Web*, 14:585–610.
13. Bareedu, Y.S. et al. (2024) Deriving semantic validation rules from industrial standards: An OPC UA study. *Semantic Web*, 15:517–554.
14. Ferranti, N. et al. (in press) Formalizing and Validating Wikidata's Property Constraints using SHACL and SPARQL. *Semantic Web*.
15. Pandit, H.J. and Esteves, B. (in press) Enhancing Data Use Ontology (DUO) for Health-Data Sharing by Extending it with ODRL and DPV. *Semantic Web*.
16. Robaldo, L. and Batsakis, S. (in press) On the interplay between validation and inference in SHACL. *Semantic Web*.
17. Otto, B. et al. (2019) Reference architecture model for international data spaces. *International Journal of Information Management*, 45:182–199.
18. Cappiello, C., Gal, A., Jarke, M., Rehof, J. and Yang, Y. (2022) Data spaces: Design, deployment, and future directions. *ACM Computing Surveys*, 55(2):Article 46.
19. Gebru, T. et al. (2021) Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.
20. Avsec, Z. et al. (2019) The Kipoi repository accelerates community exchange and reuse of predictive models for genomics. *Nature Biotechnology*, 37.
21. Baylor, D. et al. (2017) TFX: A TensorFlow-based production-scale machine learning platform. *KDD*.
22. Soldatova, L.N. and King, R.D. (2006) An ontology of scientific experiments. *Journal of the Royal Society Interface*, 3(11).
23. Correa Publio, G. et al. (2018) ML-Schema: Exposing the semantics of machine learning with schemas and ontologies.
24. Keet, C.M., Fernández, J.D. and Panov, P. (2016) MEX vocabulary: A lightweight interchange format for machine learning experiments. *ISWC*.
25. Panov, P., Dzeroski, S. and Soldatova, L.N. (2008) OntoDM: An ontology of data mining. *ICDMW*.
26. Keet, C.M. et al. (2015) The data mining optimization ontology. *Web Semantics*, 32.
27. Joachimiak, M.P. et al. (2024) The Artificial Intelligence Ontology: LLM-assisted construction of AI concept hierarchies. *Applied Ontology*.
28. Blagec, K., Barbosa-Silva, A., Ott, S. and Samwald, M. (2022) A curated, ontology-based, large-scale knowledge graph of artificial intelligence tasks and benchmarks. *Scientific Data*, 9:322.
29. Novacek, J., Ahari, A., Müller, T., Reiter, S., Viehl, A. and Bringmann, O. (2024) Ontology-Supported AI Model and Dataset Management. *IEEE 22nd International Conference on Industrial Informatics (INDIN)*, 1–6.
30. Souza, R. et al. (2019) Provenance data in the machine learning lifecycle in computational science and engineering. *WORKS*.
31. Schmidt, M. et al. (2021) FAIRness along the machine learning lifecycle using Dataverse in combination with MLflow. *ICDE Workshops*.
32. Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M. and García-Castro, R. (2022) LOT: An industrial

oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111:104755.

33. Brank, J., Grobelnik, M. and Mladenić, D. (2005) A survey of ontology evaluation techniques. *Conference on Data Mining and Data Warehouses*.
34. Poveda-Villalón, M., Gómez-Pérez, A. and Suárez-Figueroa, M.C. (2014) OOPS! (OntOlogy Pitfall Scanner!). *International Journal on Semantic Web and Information Systems*, 10(2):7–34.
35. SEMIC (2025) *MLDCAT-AP 3.0.0: Machine Learning Data Catalogue Application Profile*. European Commission.
36. SEMIC (2026) *MLDCAT-AP 3.1.0: Machine Learning Data Catalogue Application Profile*. European Commission, Candidate Recommendation, 13 May 2026.
37. International Data Spaces Association (2024) *Dataspace Protocol (DSP)*. W3C Community Draft.
38. Eclipse Dataspace Protocol (2025) *Dataspace Protocol Release 2025-1*. Eclipse Dataspace Working Group.
39. Eclipse Foundation. *Eclipse Dataspace Components (EDC)*, control-plane documentation.
40. Garijo, D. (2017) WIDOCO: A Wizard for Documenting Ontologies. *ISWC*.
41. European Union (2024) *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)*. Official Journal of the European Union.

Preguntas de competencia

R — Registro y publicación (5)

1. CQ-R1: ¿Qué modelos ha publicado un proveedor dado como activos de catálogo gobernados?
2. CQ-R2: Dado un modelo, ¿qué oferta lo registró, por qué proveedor y en qué fecha? (requiere `cadena subPropertyOf`)
3. CQ-R3: ¿Qué licencia y política de uso aplican a un modelo publicado?
4. CQ-R4: ¿Qué contrato de entrada/salida se declara para la interfaz de invocación?
5. CQ-R5: Para cada oferta, ¿qué subclase concreta de `ParticipantRole` tiene el agente proveedor? (requiere `subClassOf+`)

D — Descubrimiento y selección (4)

1. CQ-D1: ¿Qué modelos resuelven una tarea dada en un dominio dado?
2. CQ-D2: ¿Qué modelos son utilizables bajo un patrón de política concreto (por ejemplo uso no comercial)?
3. CQ-D3: ¿Qué modelos exponen un endpoint y qué método de autenticación requiere cada uno?
4. CQ-D4: ¿Qué modelos alcanzan un umbral mínimo de métrica bajo un contexto de evaluación dado?

E — Ejecución y auditabilidad (5)

1. CQ-E1: Dada una oferta, ¿qué endpoint, método de autenticación y contrato de entrada/salida aplican al modelo subyacente? (requiere `entailment`)
2. CQ-E2: Para un despliegue dado, ¿qué ejecuciones, por qué agentes y bajo qué autorización?
3. CQ-E3: Para una ejecución dada, ¿qué implementación, algoritmo, *flow* e infraestructura?
4. CQ-E4: ¿Qué evidencia de auditoría (hash, firmante, marca temporal) respalda una ejecución?
5. CQ-E5: ¿Qué artefactos derivados produjo una ejecución y bajo qué autorización?

V — Evaluación y reproducibilidad (5)

1. CQ-V1: ¿Qué contexto compartido (dataset, versión, protocolo, semilla) usa una evaluación?
2. CQ-V2: Bajo un contexto compartido, ¿qué modelo alcanza el mayor valor de métrica?
3. CQ-V3: ¿Cómo se ordenan dos o más modelos bajo el mismo contexto y métrica?
4. CQ-V4: Para un modelo dado, ¿sobre qué benchmark suites ha sido evaluado?
5. CQ-V5: ¿Qué artefactos de reproducibilidad (flow, cuaderno, tabla de resultados, checksum) respaldan una evaluación?

G — Puente de gobernanza (4, DAIMO-nativas)

1. CQ-G1: ¿Qué ofertas del catálogo federado incluyen un modelo dado?
2. CQ-G2: ¿Qué despliegues sirven un modelo dado, sobre qué infraestructura y con qué contrato de entrada/salida?
3. CQ-G3: ¿Qué autorización, y el acuerdo del que deriva, permitió una ejecución específica?
4. CQ-G4: A través de contextos de participante, ¿cuál es el bundle de procedencia completo para un artefacto derivado?

Shapes SHACL

La capa SHACL de DAIMO contiene nueve *shapes* nodales de completitud, tres *shapes* de conformidad sobre clases reutilizadas y seis invariantes SHACL-SPARQL entre clases. El módulo `shapes/daimo-shapes.ttl` se declara a su vez como `owl:Ontology` independiente con `owl:versionIRI` bajo <https://w3id.org/pionera/daimo/shapes/>.

- **Shapes de completitud (9):**
`AIAssetOfferingShape`,
`ParticipantRoleShape`,
`ModelDeploymentShape`, `IOContractShape`,
`ExecutionAuthorizationShape`,
`DerivedArtifactShape`,

SharedEvaluationContextShape,
AuditEvidenceShape y
CrossParticipantProvenanceRecordShape.

- **Shapes de conformidad sobre clases reutilizadas (3):**
OfferInDAIMOShape exige `odrl:assigner`
y `odrl:target` sobre toda `odrl:Offer`;
MachineLearningModelInDAIMOShape
exige política, título e identificador so-
bre `it6:MachineLearningModel`;
RunInDAIMOShape exige *flow*, algoritmo, agente
asociado y marca temporal sobre `it6:Run`.

- **Invariantes cruzadas (6):**
DerivationAuthorizationInvariant (INV-1),
RunAgentAuthorizationInvariant (INV-2),
DeploymentServiceInvariant (INV-3),
AuthorizationTemporalInvariant (INV-4),
OfferingPolicyTargetInvariant (INV-5) y
OfferingAssignerInvariant (INV-6); todas
como `sh:SPARQLConstraint`.

La elección de IRIs con # se limita a términos de vocabu-
lario, no a instancias operativas de alto volumen. En un espa-
cio de datos productivo, modelos concretos, datasets, ejecu-
ciones y artefactos derivados deben identificarse bajo names-
paces de los participantes o del catálogo federado; DAIMO
sólo proporciona los predicados y clases con los que esos re-
cursos se describen. Los términos nativos usan inglés ASCII:
clases en *UpperCamelCase* singular (`AIAssetOffering`,
`ModelDeployment`), propiedades en *lowerCamelCase*
verbal o relacional (`offersModel`, `authorizesRun`,
`usesEvaluationContext`) y módulos/versiones medi-
ante segmentos *slash*. Las traducciones, sinónimos y expli-
caciones se expresan con `rdfs:label`, `rdfs:comment`
y SKOS, no mediante variantes de URI.

Consultas SPARQL

Las veintitrés consultas SPARQL correspondientes
a las preguntas del Apéndice están publicadas en
`daimo/queries/queries.md`. El validador ejecuta
cada consulta sobre el cierre OWL-RL del grafo unión
ontología más ejemplo y verifica que cada una devuelve
al menos una fila. Los conteos exactos aparecen en la
Tabla 11. CQ-G2 devuelve dos filas —una por cada endpoint
del despliegue multi-protocolo REST y gRPC—, lo que
confirma que el modelado de múltiples servicios por
despliegue es un patrón legítimo y directamente observable
desde SPARQL.

Glosario de abreviaturas

Convenciones de URI

DAIMO utiliza el namespace canónico
<https://w3id.org/pionera/daimo>,
con identificadores de términos en la forma
<https://w3id.org/pionera/daimo#<Término>>
e IRIs versionadas bajo
<https://w3id.org/pionera/daimo/<versión>>,
enlazadas entre sí mediante `owl:priorVersion`.
Los módulos satélite son ontologías independientes
con IRIs versionadas propias: la capa SHACL bajo
<https://w3id.org/pionera/daimo/shapes/>
y el módulo de alineamientos bajo <https://w3id.org/pionera/daimo/align>
(este último declarando `owl:imports` sobre la ontología
base). El grafo de ejemplo utiliza el namespace local
<https://example.org/daimo-scenario/>,
claramente separado del espacio canónico. La negociación
de contenido resuelve el mismo identificador a
HTML, Turtle, RDF/XML, JSON-LD o N-Triples
en función del Accept header; la configuración
`.htaccess` correspondiente está preparada en
`w3id-redirect/.htaccess` a la espera de su
integración en `perma-id/w3id.org`.

Tabla 18. Abreviaturas utilizadas en el artículo.

Abreviatura	Significado
AI Act	Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence.
CQ	Competency Question.
DAIMO	Data-space AI Model Ontology.
DCAT	Data Catalog Vocabulary.
DCAT-AP	DCAT Application Profile.
DL	Description Logic.
DOI	Digital Object Identifier.
DPV	Data Privacy Vocabulary.
DSP	Dataspace Protocol.
EDC	Eclipse Dataspace Components.
FAIR	Findable, Accessible, Interoperable, Reusable.
LOT	Linked Open Terms.
MLDCAT-AP	Machine Learning Data Catalogue Application Profile.
ODRL	Open Digital Rights Language.
OOPS!	OntOlogy Pitfall Scanner.
OWL	Web Ontology Language.
PROV-O	PROV Ontology.
RDF	Resource Description Framework.
RDFS	RDF Schema.
SHACL	Shapes Constraint Language.
SKOS	Simple Knowledge Organization System.
SPARQL	SPARQL Protocol and RDF Query Language.
SWJ	Semantic Web Journal.
TTL	Turtle.
WIDOCO	Wizard for Documenting Ontologies.